FACULTY OF TECHNOLOGICAL SCIENCES AND INNOVATION

Master's Degree Programme in Computer Security (LM-66)

Final Thesis

Secure Software Development Lifecycle

A Structured Review of Principles, Practices, and Lifecycle Security Controls

SUPERVISOR

Prof. Davide Berardi

CANDIDATE

Patrik Brighenti

Student ID: 0662400360

Academic Year 2025/2026

Contents

Introduction **vii**

Background and Motivation vii

Secure Software Development Lifecycle viii

Methodological Approach ix

Scope and Limitations x

Structure of the Thesis xi

1 Foundations and Security Requirements in the Software Lifecycle **1**

1.1 Fundamentals of Secure Software Engineering 1

1.2 Core Security Principles 3

1.2.1 Confidentiality 3

1.2.2 Integrity 3

1.2.3 Availability 4

1.2.4 Non-Repudiation and Accountability 4

1.2.5 Interdependencies Among Security Principles 4

1.3 Risk-Oriented Approaches to Software Security 5

1.3.1 Security as Risk Management 6

1.3.2 Data Classification and Impact Analysis 6

1.3.3 Threat Modeling and Misuse Perspectives 6

1.3.4 Balancing Risk, Cost, and Security Objectives 7

1.4 Security Requirements 7

1.4.1 Elicitation of Security Requirements 8

1.4.2 Misuse and Abuse Cases as Requirement Drivers 8

1.4.3 Analysis and Prioritization of Security Requirements . . 8

1.4.4 Validation and Traceability of Security Requirements . . 9
1.5 Regulatory and Compliance Considerations 10
1.5.1 Regulatory Requirements as Security Drivers 10
1.5.2 Data Protection, Privacy, and Information Sensitivity . . 10
1.5.3 Compliance Standards and Frameworks 11
1.5.4 Intellectual Property and Contractual Obligations 11
1.5.5 Integrating Compliance into Security Requirements . . . 12
1.6 Integrating Security Requirements into Early Lifecycle Stages . 12
1.6.1 Security by Design and Proportional Protection 12
1.6.2 Architectural Simplicity and Control Enforcement . . . 13
1.6.3 Shared Mechanisms, Reuse, and Risk Concentration . . 13
1.6.4 Fail-Safe Defaults and Human Factors 13
1.6.5 Continuity Across the Software Lifecycle 14

2 Secure Design, Implementation, and Verification Practices 15
2.1 Security-Focused Architectural Design 15
2.1.1 Threat Modeling Methodologies 16
2.1.2 Architectural Risk Analysis 19
2.1.3 Secure Design Patterns and Anti-Patterns 21
2.2 Secure Implementation Practices 22
2.2.1 Secure Coding Principles 23
2.2.2 Input Validation, Error Handling, and Data Protection . . 24
2.2.3 Code Review Processes 25
2.2.4 Static Analysis and Automated Code Scanning 26
2.3 Security Testing Across the Lifecycle 28
2.3.1 Security Test Planning 28
2.3.2 SAST, DAST, SCA, and Fuzzing 30
2.3.3 Verification of Implemented Security Controls 32
2.4 Configuration and Change Management 33
2.4.1 Secure Build Pipelines 34
2.4.2 Dependency and Configuration Security 36
2.4.3 Release and Version Control Considerations 38

3 Deployment, Operational Security, and Software Supply Chain Integrity **41**

3.1 Secure Deployment and Environment Hardening 41

3.2 Operational Security Controls 46

3.3 Software Supply Chain and Third-Party Component Security . . 51

3.4 Secure Maintenance and Continuous Improvement 57

3.5 Secure Decommissioning and Disposal of Software Components 63

3.6 Synthesis: Integrating All Lifecycle Controls into a Consistent SSDLC Model 68

Conclusion **73**

Summary 73

Benefits and Limitations of Current SSDLC Practices 75

Emerging Trends 77

Future Research Directions 79

List of Figures

1.1 Core security principles and their interdependencies. 5
1.2 Risk-oriented approach to software security. 7
1.3 Security requirements across the software lifecycle. 9

2.1 Threat modeling workflow embedded in architectural design. . . 17
2.2 Example of trust boundaries and controlled data flows. 18
2.3 Illustrative matrix for architectural risk prioritization. 20
2.4 Secure implementation pipeline with local checks, peer review, and CI security gates. 24
2.5 Input validation and exception handling flow. 25
2.6 Illustrative comparison of manual review and automated scanning coverage. 27
2.7 Risk-based workflow for security test planning across the lifecycle. 29
2.8 Indicative mapping of major security testing techniques to lifecycle stages and weakness classes. 31
2.9 Traceability loop for verifying implemented security controls. . 33
2.10 Secure build pipeline with policy gates and supporting controls. 35
2.11 Configuration baseline and change control loop for secure software delivery. 38
2.12 Release and version-control integrity chain from reviewed change to deployment decision. 40

3.1 Secure deployment promotion flow 42
3.2 Environment hardening layers 43

3.3 Deployment hardening control matrix 45
3.4 Operational monitoring and logging pipeline 47
3.5 Risk-based vulnerability and patch prioritization 49
3.6 Incident response feedback loop 50
3.7 Dependency evaluation workflow 53
3.8 SBOM lifecycle and transparency loop 54
3.9 Third-party integration risk map 56
3.10 Secure maintenance cycle 58
3.11 Vulnerability remediation decision flow 60
3.12 Continuous improvement metrics loop 62
3.13 Retirement and decommissioning workflow 64
3.14 Preservation and sanitization decision flow 66
3.15 Integrated SSDLC control model 69
3.16 Lifecycle control mapping matrix 70
3.17 Lifecycle assurance feedback loop 71

Introduction

Background and Motivation

In recent decades, software systems have become a foundational component of modern digital societies. Organizations increasingly rely on software to manage critical processes, deliver services, and process sensitive information. As a consequence, the reliability and security of software systems directly influence organizational continuity, economic stability, and public trust.

At the same time, the growing complexity of software architectures has significantly expanded their exposure to security threats. Modern software systems are frequently distributed, interconnected, and continuously evolving. The widespread adoption of cloud computing, agile development practices, and third-party software components has accelerated development cycles but has also introduced new vulnerabilities and dependencies. Security incidents affecting software systems have demonstrated that weaknesses in design or implementation can lead to data breaches, service disruptions, financial losses, and long-term reputational damage.

A recurring characteristic of many security incidents is that their root causes can be traced back to deficiencies introduced during the early phases of the software lifecycle. Security is often addressed as a secondary concern, subordinated to functional requirements, performance goals, or delivery deadlines. When vulnerabilities are discovered at later stages—such as during deployment or operation—their remediation typically requires significant effort and may involve architectural changes that are difficult to implement without disrupting business processes (**Kissel et al., 2008**).

These observations have highlighted the limitations of reactive security approaches that focus primarily on perimeter defenses or post-deployment controls. Instead, there is a growing consensus that software security must be considered as an integral aspect of the entire development lifecycle (**Cybersecurity and Infrastructure Security Agency, 2023a; Souppaya et al., 2022**). This perspective emphasizes the need to incorporate security considerations from the earliest stages of requirements definition and design, and to maintain them throughout implementation, testing, deployment, operation, and maintenance.

The motivation of this thesis arises from the need to systematize and clarify how security can be effectively integrated across the software development lifecycle. While numerous guidelines and frameworks propose security-oriented practices, they are often addressed to specific technical audiences or presented in a fragmented manner. This work aims to provide a structured and coherent overview of lifecycle-based software security, highlighting its relevance from both a technical and organizational standpoint.

Secure Software Development Lifecycle

Integrating security throughout the software development lifecycle represents a fundamental requirement for building resilient and trustworthy systems. From a risk management perspective, addressing security concerns early allows organizations to identify potential threats and vulnerabilities before they are embedded into system architectures or business processes. Early mitigation reduces the likelihood of critical failures and limits the potential impact of security incidents.

Economic considerations further reinforce the importance of this approach. The cost of correcting security flaws increases significantly as development progresses (**Kissel et al., 2008**). Vulnerabilities detected during requirements analysis or design can often be addressed through relatively minor adjustments, whereas flaws discovered after deployment may require extensive reengineering, emergency patches, or operational workarounds. Consequently, lifecycle-oriented security contributes to cost efficiency and long-term sustainability.

From an organizational perspective, embedding security into the lifecycle promotes alignment between technical teams, management, and governance structures. Security requirements can be explicitly defined, documented, and traced throughout development activities, facilitating accountability and decision-making. This alignment supports a proactive security culture in which responsibilities are clearly assigned and security objectives are integrated into project planning and execution.

Regulatory and compliance considerations also play a significant role. Legal frameworks governing data protection, information security, and service reliability increasingly require organizations to demonstrate systematic and preventive approaches to security. Isolated technical controls are no longer sufficient to meet these expectations. Instead, organizations are expected to adopt structured processes that ensure security is consistently addressed across all phases of system development and operation.

Moreover, the importance of lifecycle security is amplified by the extensive use of third-party software components and external services. Dependencies on external libraries, platforms, and providers introduce risks that cannot be managed solely at deployment time. A lifecycle-based approach enables organizations to assess, monitor, and manage these risks continuously, ensuring that security considerations extend beyond internally developed components.

Methodological Approach

This thesis adopts a qualitative and descriptive methodological approach aimed at organizing and synthesizing established knowledge related to secure software development. Rather than conducting an extensive systematic review of academic literature, the work focuses on the analysis of authoritative reference materials and consolidated industry guidelines that reflect current professional practices in software security.

The primary sources considered include technical manuals, security frameworks, and official guidelines published by recognized institutions and organi-

zations operating in the field of cybersecurity and secure software engineering (**Cybersecurity and Infrastructure Security Agency, 2023a; OWASP Foundation, 2026d; Ross et al., 2022; Souppaya et al., 2022**). These sources provide structured and practical perspectives on how security can be integrated across the software lifecycle. Selected external resources are used to complement these references where appropriate, in order to clarify specific concepts or contextualize particular practices.

The analysis is organized according to the phases of the software development lifecycle, including requirements definition, design, implementation, testing, deployment, operation, and maintenance. For each phase, the discussion highlights relevant security objectives, principles, and controls, emphasizing their role within a lifecycle-oriented security model. The approach is intentionally technology-agnostic, avoiding detailed implementation guidance or tool-specific solutions.

The objective of this methodology is to provide a coherent conceptual framework rather than to introduce new empirical findings. The thesis aims to support understanding and critical reflection on existing practices, making it suitable for an academic context focused on systematization and structured analysis within the domain of cybersecurity.

Scope and Limitations

The scope of this thesis is limited to the examination of security principles and practices applicable to the software development lifecycle. The discussion focuses on general-purpose software systems developed and operated within organizational environments and does not address specialized domains such as hardware security, embedded systems, or safety-critical real-time software.

The thesis does not provide detailed technical instructions for specific programming languages, platforms, or development tools. Instead, it adopts a high-level and conceptual perspective that emphasizes principles and processes applicable across different technological contexts. While this enhances general

applicability, it necessarily limits the depth of technical detail in certain areas.

Another limitation concerns the absence of empirical validation. The proposed lifecycle-oriented framework is derived from established references and widely adopted practices, but it is not evaluated through case studies or quantitative analysis. Future research could extend this work by applying the framework to specific organizational scenarios and assessing its effectiveness in practice.

Finally, cybersecurity is a rapidly evolving field. New technologies, development paradigms, and threat vectors continuously reshape the software security landscape, and some practices discussed in this thesis may require adaptation over time. Nevertheless, the underlying principles of early integration, continuous risk management, and lifecycle-wide security governance remain broadly applicable.

Structure of the Thesis

This thesis is organized into three main chapters, framed by the present introduction and the concluding section.

The first chapter establishes the theoretical foundations of secure software engineering by introducing core security principles, risk-oriented approaches, and the role of security requirements within the software development lifecycle.

The second chapter examines security practices applied during design, implementation, and verification phases, with particular emphasis on architectural considerations, secure coding principles, and security testing activities.

The third chapter focuses on the later stages of the lifecycle, addressing deployment, operational security, maintenance processes, and software supply chain considerations, with the aim of presenting an integrated and consistent security perspective.

The concluding section summarizes the main findings of the thesis, discusses the strengths and limitations of current Secure Software Development Lifecycle practices, and outlines possible directions for future research.

Chapter 1

Foundations and Security Requirements in the Software Lifecycle

1.1 Fundamentals of Secure Software Engineering

Secure software engineering extends traditional software engineering by treating security as a core quality attribute rather than as a late corrective activity. Whereas conventional development often prioritizes functional correctness, performance, and usability, secure software engineering also addresses asset protection, adversarial misuse, and the resilience of software under hostile conditions. Because contemporary systems operate in distributed and interconnected environments, security concerns must be integrated across the entire software lifecycle (**Ross et al., 2022**).

A fundamental distinction exists between functional and security requirements. Functional requirements describe the services a system must provide under expected conditions, while security requirements define constraints that preserve acceptable behavior in the presence of faults, misuse, or malicious actions. Security properties such as confidentiality, integrity, and availability are therefore cross-cutting: they influence architecture, data handling, interfaces, and operational workflows. For this reason, security cannot be reduced to iso-

lated controls added after development, but must be addressed systematically from the beginning.

Identity, access control, and accountability form the operational basis of secure software systems. Each interaction can be understood as a subject performing an action on an object, and security depends on correctly establishing identity, authorizing permitted actions, and recording relevant events. Authentication confirms who a subject is, authorization defines what that subject may do, and accountability makes those actions auditable. Without a coherent identity and authorization model, higher-level security policies are difficult to enforce consistently.

Secure software engineering is also inherently risk-oriented. Absolute protection is neither realistic nor economically sustainable; instead, the objective is to reduce risk to an acceptable level (**2022**). Security decisions must therefore be guided by the value of protected assets, the plausibility of threats, and the impact of compromise. This view supports prioritization, proportionality, and explicit trade-offs rather than the indiscriminate accumulation of controls.

Closely related to this perspective is software assurance, understood as the degree of confidence that software behaves as intended and is sufficiently resistant to vulnerabilities throughout its operational life (**Souppaya et al., 2022**). Assurance is built through disciplined development practices, review, and verification. It is not a binary condition, but a level of justified confidence that varies according to system criticality and the rigor of the applied controls.

Finally, secure software engineering places strong emphasis on early lifecycle decisions. Requirements and architectural choices shape trust boundaries, data flows, exposure surfaces, and feasible control models. When security is considered only during implementation or testing, remediation is typically more expensive and less effective (**Kissel et al., 2008**). Early integration therefore provides the foundation for the design, implementation, and verification practices examined in the following chapters.

1.2 Core Security Principles

Security requirements are derived from a set of foundational principles that define the protection goals of a software system independently of specific technologies. These principles provide a conceptual basis for reasoning about threats, controls, and acceptable system behavior (**Ross et al., 2022**). The most widely recognized are confidentiality, integrity, and availability, complemented by non-repudiation and accountability.

1.2.1 Confidentiality

Confidentiality concerns the prevention of unauthorized disclosure of information. In software systems, it requires that data be accessible only to authorized subjects under defined conditions. Authorization is therefore central to confidentiality, since it determines which actors may access which resources and under what constraints.

From an engineering perspective, confidentiality must be considered across the full data lifecycle. Data at rest requires protection against unauthorized reading or extraction, data in transit must be protected from interception, and data in use can be exposed through application state, memory, or runtime interactions. These distinctions directly affect architecture, interface design, and the placement of security controls.

1.2.2 Integrity

Integrity refers to protection against unauthorized or unintended modification of data and system state. Its objective is not only to prevent improper change, but also to preserve correctness and trustworthiness. In many domains, including finance, healthcare, and control systems, integrity is as critical as confidentiality because altered information can lead to unsafe or misleading outcomes.

In practical terms, integrity requirements regulate which operations are allowed on data and how those operations are validated. They also support the detection of alteration through mechanisms such as checks, controlled updates,

versioning, and process controls. Integrity therefore influences validation logic, workflows, and the structure of update mechanisms, not merely low-level implementation details.

1.2.3 Availability

Availability requires that systems and services remain usable for authorized users when needed. It is associated with resilience against failures, overload, and deliberate disruption, but also with the controlled use of resources so that unauthorized actors cannot exhaust them. Availability is therefore both a reliability and a security concern.

At design level, availability leads to decisions about redundancy, recovery, fault tolerance, and graceful degradation. These decisions often involve trade-offs with other objectives, since increasing resilience may introduce complexity or new attack surfaces. Availability must therefore be evaluated in relation to broader risk and system context rather than treated in isolation.

1.2.4 Non-Repudiation and Accountability

Non-repudiation ensures that subjects cannot credibly deny actions they have performed, while accountability requires that those actions be associated with identifiable subjects in a reliable manner. Together, these principles support auditability, incident analysis, and organizational trust in system records.

In practice, they depend on the combined use of identity management, authorization, and logging. From a software engineering standpoint, these requirements influence event design, traceability, and retention policies. They must also be balanced against privacy and legal constraints, especially in systems processing sensitive personal or organizational data.

1.2.5 Interdependencies Among Security Principles

Although these principles are often presented separately, they are closely interdependent. Measures that strengthen one objective may affect another: ac-

countability may require additional data collection, redundancy may complicate integrity management, and restrictive confidentiality measures may reduce availability or usability. Effective security engineering therefore requires a holistic view that considers how principles interact within the same system and risk context. Figure 1.1 illustrates these relationships and the supporting role of non-repudiation and accountability.

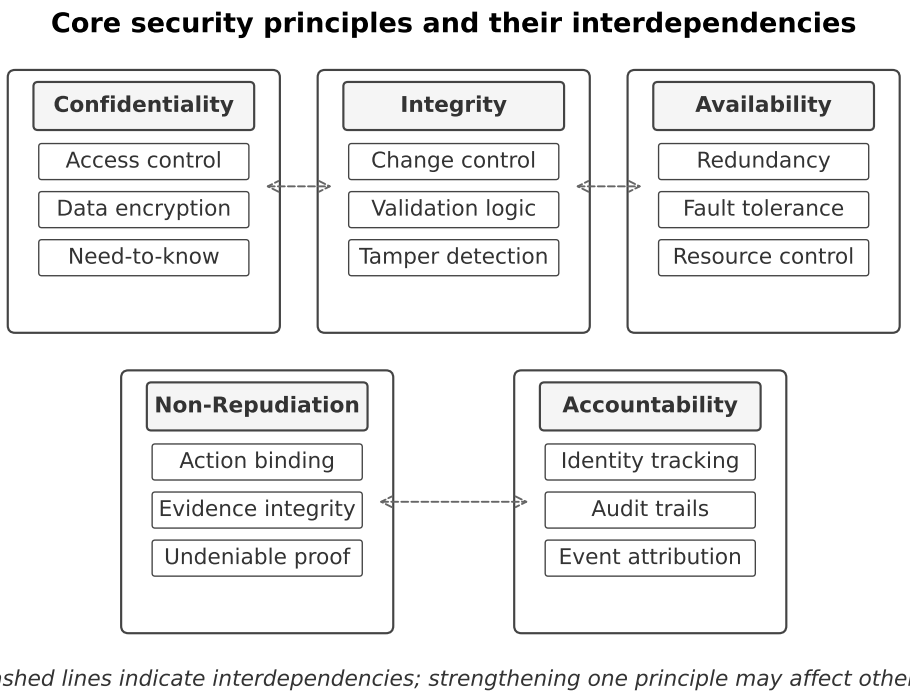

Figure 1.1: Core security principles and their interdependencies.

1.3 Risk-Oriented Approaches to Software Security

Modern software security is best understood as a structured form of risk management (**Kissel et al., 2008; Ross et al., 2022**). Rather than promising complete protection, it aims to reduce the probability and consequences of harmful events to an acceptable level. In this sense, risk links threats, vulnerabilities, and impact, providing a practical basis for deciding which controls deserve priority.

1.3.1 Security as Risk Management

A risk-oriented approach aligns security efforts with business and operational priorities. Different assets have different value, and not all threats justify the same investment. By identifying critical assets, plausible threat sources, and likely consequences, organizations can focus security resources where failure would matter most.

Risk management is also continuous. As software evolves, new features, dependencies, and deployment contexts change the attack surface. Security assessment must therefore be revisited throughout the lifecycle rather than treated as a one-time planning exercise.

1.3.2 Data Classification and Impact Analysis

A practical instrument of risk-oriented security is data classification. Classifying information according to sensitivity and business impact allows teams to apply protection measures proportionate to the consequences of compromise. High-impact data generally demands stronger confidentiality, integrity, and availability safeguards than routine operational data.

Impact analysis should consider more than direct financial loss. Operational disruption, legal liability, reputational harm, and consequences for individuals may all justify stronger controls. Data classification is also closely linked to the data lifecycle, since creation, storage, transmission, retention, and disposal each introduce different exposure points and protection requirements.

1.3.3 Threat Modeling and Misuse Perspectives

Risk-oriented analysis requires attention not only to intended use, but also to misuse (**OWASP Foundation, 2025n**). Functional models describe how a system should behave under normal conditions; misuse and abuse perspectives instead consider how it may be attacked, manipulated, or incorrectly used. This shift is essential for identifying security-relevant failure modes early.

Adopting an attacker-oriented view helps reveal unauthorized actions, sen-

sitive data exposure, and policy violations that might remain invisible in purely functional analysis. Such scenarios may involve external attackers, malicious insiders, careless users, or adverse environmental conditions. Framing them explicitly improves risk identification and supports more focused requirements and controls.

1.3.4 Balancing Risk, Cost, and Security Objectives

A risk-based approach also makes trade-offs explicit. Stronger controls may improve protection, but they can also increase complexity, cost, and operational burden. Security decisions must therefore be proportionate to asset value and realistic threat levels. Overprotection can waste scarce resources, while underprotection leaves high-impact assets exposed. Risk-oriented security provides a rational framework for navigating these choices and for justifying design decisions. Figure 1.2 summarizes this iterative process, from asset identification through control selection to residual risk acceptance.

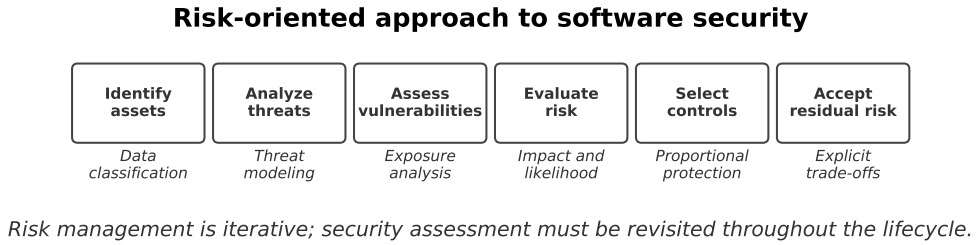

Figure 1.2: Risk-oriented approach to software security.

1.4 Security Requirements

Security requirements translate broad protection goals and risk findings into specific, verifiable expectations for system behavior (**Kissel et al., 2008; Souppaya et al., 2022**). Unlike functional requirements, they define constraints that preserve acceptable operation in the presence of attack, misuse, or failure. They

therefore connect high-level principles with concrete design, implementation, and verification activities.

1.4.1 Elicitation of Security Requirements

Elicitation begins with identifying what must be protected and why. Relevant inputs include business objectives, critical assets, risk assessments, organizational policies, and legal obligations. Security requirements are often derived from goals such as confidentiality, integrity, availability, and accountability, but they must be refined into operational statements that can guide engineering decisions.

Effective elicitation is interdisciplinary. Developers, architects, security specialists, and business stakeholders all contribute perspectives necessary to define requirements that are both technically meaningful and aligned with organizational needs.

1.4.2 Misuse and Abuse Cases as Requirement Drivers

Traditional use cases describe legitimate interactions with the system; misuse and abuse cases complement them by modeling harmful or unauthorized behavior. This makes them particularly valuable for discovering security requirements that would not emerge from functional analysis alone.

By exploring how attackers, insiders, or even ordinary users under abnormal conditions might cause harm, teams can derive requirements aimed at prevention, detection, and mitigation. Misuse cases also improve communication by translating abstract risks into concrete scenarios with visible consequences.

1.4.3 Analysis and Prioritization of Security Requirements

Once identified, security requirements must be analyzed for feasibility, consistency, and relative importance. Not all requirements carry the same weight, and attempting to implement every possible control without prioritization can introduce unnecessary complexity. Risk therefore remains the main criterion for sequencing effort.

Prioritization should consider threat likelihood, impact, data sensitivity, regulatory obligations, and possible conflicts with usability, performance, or maintainability. This structured analysis helps keep requirements realistic and aligned with overall system objectives.

1.4.4 Validation and Traceability of Security Requirements

Security requirements must also be validated and traced across the lifecycle (**2022**). Validation confirms that they reflect real protection goals and stakeholder needs, while traceability links them to sources, design choices, implementation artifacts, and verification activities.

Traceability is especially important when systems evolve. Changes in scope, architecture, or threat conditions may require requirements to be updated or reinterpreted. Maintaining traceability supports impact analysis, facilitates auditing, and provides evidence that security has been addressed systematically rather than informally.

In this sense, security requirements are not static statements but managed lifecycle artifacts. Their value lies not only in being written down, but in being preserved, refined, and verified as the system changes. Figure 1.3 shows how elicitation, analysis, specification, and validation form an iterative process underpinned by traceability.

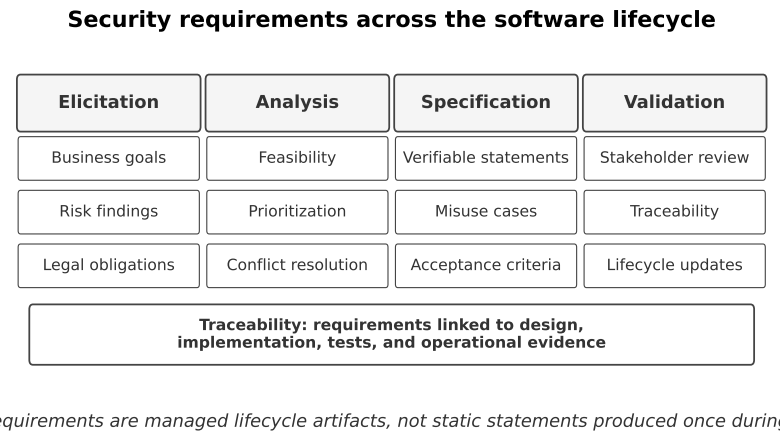

Figure 1.3: Security requirements across the software lifecycle.

1.5 Regulatory and Compliance Considerations

Regulatory and compliance obligations are major external drivers of secure software development. Unlike internally defined security goals, they originate from laws, contracts, sectoral rules, and audit expectations. Because they constrain data handling, access management, logging, retention, and operational processes, they must be considered during requirements definition rather than after design decisions have already been made.

A useful distinction exists between risk-driven security and compliance-driven security. Risk analysis aims to reduce the likelihood and impact of harmful events, whereas compliance seeks conformity with predefined obligations. Compliance alone does not guarantee effective security, yet non-compliance can lead to sanctions, contractual disputes, and reputational damage. Mature software engineering must therefore reconcile both perspectives.

1.5.1 Regulatory Requirements as Security Drivers

Many regulatory regimes impose explicit expectations regarding confidentiality, access control, auditing, incident handling, and records management. In regulated sectors these obligations function as mandatory constraints, regardless of whether local risk assessments would otherwise justify them. Development teams must therefore translate them into concrete and testable system requirements.

The challenge becomes greater when software must satisfy multiple overlapping frameworks. Different jurisdictions or contractual environments may impose partially different obligations, requiring careful interpretation and prioritization during requirements engineering.

1.5.2 Data Protection, Privacy, and Information Sensitivity

Privacy and data protection obligations are especially relevant because they affect the full information lifecycle. Requirements such as data minimization, purpose limitation, retention limits, and lawful processing have direct implications for

system design, storage models, and user-facing functionality. In this way, privacy compliance reinforces the need for data classification and impact analysis.

Systems that process personally identifiable or otherwise sensitive information usually require stronger safeguards and clearer accountability for how data is collected, stored, transferred, and deleted. These are not peripheral issues, but central design constraints.

1.5.3 Compliance Standards and Frameworks

Beyond statutory rules, standards and frameworks provide structured guidance for security management and software assurance (**OWASP Foundation, 2026d; Souppaya et al., 2022**). Even when formally voluntary, they are often embedded in contracts, procurement requirements, or external assessments. For organizations, they offer a common language for defining expectations and demonstrating due diligence.

However, standards are broad by nature. They support consistency and repeatability, but they do not replace context-specific analysis. Their value lies in guiding secure practice, not in substituting for system-specific engineering judgment.

1.5.4 Intellectual Property and Contractual Obligations

Compliance considerations also include intellectual property protection and contract-specific duties. Access restrictions, confidentiality protections, licensing terms, service-level commitments, and incident notification clauses can all shape software requirements. These obligations may impose security-relevant duties even where no direct statutory rule applies.

For this reason, contractual analysis should be treated as part of requirements elicitation. A failure to satisfy contractual security expectations may create liability even if no formal legal breach has occurred.

1.5.5 Integrating Compliance into Security Requirements

Compliance is most effective when integrated into the general security requirements process. Obligations should be identified early, mapped to assets and processes, and expressed as verifiable requirements. Addressing them only after architecture or implementation decisions have been made often causes costly redesign and incomplete mitigation.

At the same time, compliance should not replace risk analysis. It establishes a baseline, but resilient systems still require context-aware controls that respond to actual threats and operational realities. Secure software engineering therefore treats compliance and risk management as complementary rather than competing perspectives.

1.6 Integrating Security Requirements into Early Lifecycle Stages

The point at which security requirements are introduced strongly influences their effectiveness. Decisions made during requirements analysis and architectural design shape the structure of trust boundaries, access paths, data flows, and control enforcement. Early integration therefore reduces structural weaknesses and limits the cost of correction later in the lifecycle.

1.6.1 Security by Design and Proportional Protection

Early lifecycle integration reflects the broader idea of security by design (**Cybersecurity and Infrastructure Security Agency, 2023a**): security should be built into the system as an inherent property rather than attached as a corrective layer. This does not imply maximizing protection without limit. A more realistic objective is proportional protection, meaning that controls should be commensurate with system criticality, exposure, and acceptable risk.

Establishing proportionality early helps teams avoid both underprotection and unnecessary complexity. It also supports explicit trade-offs between security

and other quality attributes such as usability, performance, and maintainability.

1.6.2 Architectural Simplicity and Control Enforcement

Early design is also the stage at which simplicity can still be preserved. Architectures with fewer unnecessary components, interfaces, and services are easier to analyze, test, and secure. Conversely, excessive complexity creates hidden interactions and makes control enforcement inconsistent.

At this stage, requirements related to mediation and policy enforcement should also be reflected in the architecture. If access control paths and trust boundaries are not defined early, later attempts to impose consistent checks are often incomplete or fragile.

1.6.3 Shared Mechanisms, Reuse, and Risk Concentration

Architectural decisions determine how mechanisms and components are shared. Excessive sharing can create channels for unintended information flow or increase the impact of failures. Reducing unnecessary common mechanisms therefore helps contain risk.

At the same time, modern development relies heavily on reuse. Reusing established components can improve reliability and accelerate delivery, but it also introduces dependency, supply-chain, and inherited-vulnerability risks. Early integration of security requirements allows these trade-offs to be assessed before they become embedded in the system.

1.6.4 Fail-Safe Defaults and Human Factors

Secure design also requires safe default behavior. Systems should move toward a controlled and secure state when failures or unexpected conditions occur, rather than exposing data or bypassing checks. Defining these expectations early improves predictability and reduces the risk of insecure exception paths.

Human factors matter as well. Security mechanisms that are unnecessarily obscure or disruptive often encourage bypasses and workarounds. Usable con-

trols are therefore not secondary to security, but one of its conditions.

1.6.5 Continuity Across the Software Lifecycle

Early integration does not mean that security belongs only to the initial phases of development. Instead, it establishes the assumptions, requirements, and architectural constraints that guide later implementation, testing, deployment, and operations. These early decisions must remain visible and traceable as the system evolves.

In summary, integrating security requirements early creates a stable basis for the rest of the lifecycle. Proportional protection, simplicity, controlled reuse, safe defaults, and continuity of decisions all contribute to systems that are more robust, maintainable, and easier to verify in subsequent stages.

Chapter 2

Secure Design, Implementation, and Verification Practices

2.1 Security-Focused Architectural Design

Architectural design is the point in the software lifecycle where many security properties become structurally constrained. At this stage, the system is not yet defined by source code, but by components, interfaces, trust assumptions, and interaction rules. Decisions concerning service decomposition, data movement, authentication boundaries, error propagation, and dependency placement can either reduce or amplify future exposure. For this reason, secure architectural design should not be treated as a generic quality discussion; it should be approached as a deliberate activity in which security objectives are translated into technical structures that can later be implemented and verified. NIST's systems security engineering guidance presents architectural design as a primary means of building trustworthy systems (**Ross et al., 2022**). CISA's secure-by-design guidance makes the same point from a product perspective, arguing that security should be embedded into design decisions rather than appended after those decisions have solidified (**Cybersecurity and Infrastructure Security Agency, 2023a**).

A security-focused architecture does more than list controls. It clarifies what must be protected, who or what is trusted, which assumptions are acceptable,

and how failures should be contained. In practice, this requires a combination of threat-oriented reasoning, architectural risk analysis, and the disciplined use of secure patterns. These activities are complementary. Threat modeling helps identify plausible abuse paths (**National Institute of Standards and Technology, 2016; OWASP Foundation, 2025n**); architectural risk analysis helps prioritize architectural weaknesses according to likely impact (**Kazman et al., 2000; Ross et al., 2022**); and secure design patterns provide reusable structures for implementing security requirements consistently (**OWASP Foundation, 2025j**). The following subsections examine these three dimensions in order to show how security can be embedded into architecture before detailed implementation begins.

2.1.1 Threat Modeling Methodologies

Threat modeling is a structured method for identifying how a system might be attacked, misused, or made to fail from a security perspective. Its main value lies in timing: when performed during architectural design, it allows security decisions to be made before vulnerabilities become expensive design constraints. Rather than focusing on isolated coding defects, threat modeling considers the logical structure of the system, the actors that interact with it, the assets being handled, the boundaries between trust zones, and the consequences of unauthorized operations. OWASP describes threat modeling as a design-phase activity aimed at building security into a system rather than appending it afterward (**2025n**). NIST, by contrast, characterizes it as a form of risk assessment applicable to data, applications, hosts, and broader system environments (**National Institute of Standards and Technology, 2016**).

A practical threat-modeling exercise usually begins with system decomposition. Designers identify major components, external actors, entry points, privileged functions, data stores, and communication channels. This decomposition is not merely descriptive. It enables analysts to ask focused questions: where is identity established, where is data transformed, where are authorization decisions enforced, and where can trust be subverted? In architectures composed

of multiple services or external integrations, the mapping of trust boundaries becomes especially important because attacks often emerge where assumptions cross administrative, network, or privilege domains. Figure 2.1 summarizes an iterative workflow in which security objectives inform system decomposition, threat identification, and the subsequent revision of architectural controls.

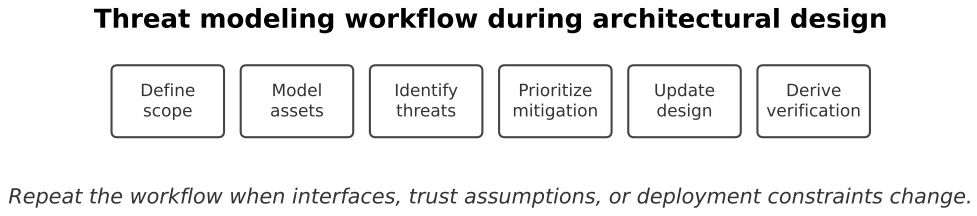

Figure 2.1: Threat modeling workflow embedded in architectural design.

Among the most widely used methodologies, STRIDE remains particularly useful because it organizes reasoning around recurring categories of threats: spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege. The method is valuable not because it guarantees completeness, but because it imposes a disciplined lens on each component and data flow. Microsoft documentation preserves this logic by linking threat discovery to the creation of a system diagram, the generation of candidate threats, the definition of mitigations, and the validation of those mitigations during design refinement (**Microsoft, 2022**). A different but compatible perspective is provided by NIST's data-centric approach, which places emphasis on how sensitive data is created, processed, transmitted, stored, and destroyed. This is especially useful when confidentiality, privacy, or integrity requirements are the primary security drivers (**National Institute of Standards and Technology, 2016**).

The technical quality of threat modeling depends heavily on the quality of the architectural representation being reviewed. Generic block diagrams are often insufficient, because they hide the exact interfaces through which trust is established and enforcement occurs. A more security-relevant model should highlight where requests originate, where they are validated, how identity is propagated, where secrets are retrieved, and which components interact across different trust

zones. Figure 2.2 illustrates this point through a simple architecture in which an external zone, an application zone, and a data zone are separated by explicit trust boundaries. Even in a simplified representation, the diagram makes visible several questions that can guide design review: whether all external traffic terminates at a controlled gateway, whether token validation is centralized, whether secrets are retrieved through a dedicated service, and whether the application can reach logging and data services without bypassing policy enforcement.

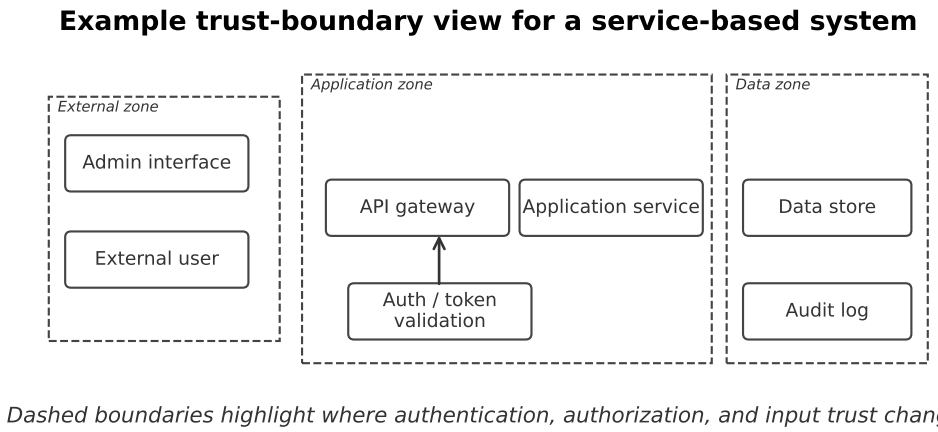

Figure 2.2: Example of trust boundaries and controlled data flows.

An important methodological point is that threat modeling should be iterative rather than ceremonial. If performed once and archived, it quickly loses value. Architectural changes, new dependencies, additional interfaces, and altered deployment assumptions can invalidate earlier conclusions. Effective practice therefore connects threat modeling to design reviews, major change requests, and pre-release verification checkpoints. This view is consistent with CISA's secure-by-design recommendations, which emphasize making security decisions explicit and revisiting them whenever the architecture evolves (**Cybersecurity and Infrastructure Security Agency, 2023a**). OWASP's current material on insecure design reinforces the same lesson by treating design decisions themselves as a recurring source of serious security failure (**OWASP Foundation, 2025b**). Threat modeling can thus be understood as the bridge between abstract security requirements and the concrete architectural measures that later guide secure implementation and testing.

2.1.2 Architectural Risk Analysis

Threat modeling identifies what might go wrong; architectural risk analysis helps determine which design issues matter most and why. In complex systems, the relevant problem is rarely the existence of isolated threats, but the concentration of exposure created by specific structural choices. For example, an architecture in which multiple services share broad privileges may remain functional, yet still create a high-impact failure mode if one compromised component can move laterally across the environment. Architectural risk analysis therefore evaluates design choices in terms of exploitability and impact, but also in terms of how they interact with other quality attributes such as performance, availability, and maintainability. That perspective is visible both in scenario-based architecture evaluation methods such as ATAM (**Kazman et al., 2000**) and in NIST's systems security engineering guidance (**Ross et al., 2022**).

A useful perspective here is provided by scenario-based architecture evaluation methods such as ATAM. The central idea is that architectural properties become visible when the system is examined through concrete scenarios rather than abstract principles alone. Security scenarios may describe, for instance, a malicious user attempting to access another tenant's data, an attacker trying to abuse a misconfigured interface, or an internal component failing in a way that weakens authorization enforcement. ATAM is not exclusively a security method, but it is highly valuable because it reveals sensitivity points, trade-offs, and risks associated with the architecture under realistic usage and failure conditions (**Kazman et al., 2000**). This confirms that security should be assessed as a system property that interacts with other architectural concerns rather than as an isolated checklist.

In practical terms, architectural risk analysis often focuses on a recurring set of questions. Which components are exposed to untrusted input? Where are privileged operations concentrated? Which services hold security-critical data or secrets? What dependencies could create systemic failure if compromised? How is failure contained when a control is bypassed? These questions support the prioritization of design weaknesses before implementation begins. Figure 2.3

provides an illustrative matrix for assigning priority to architectural risks on the basis of exploitability and impact. The model is intentionally simple, but it reflects a useful principle: design weaknesses that combine broad exposure with severe business or mission impact should be treated as early architectural priorities rather than deferred as implementation details.

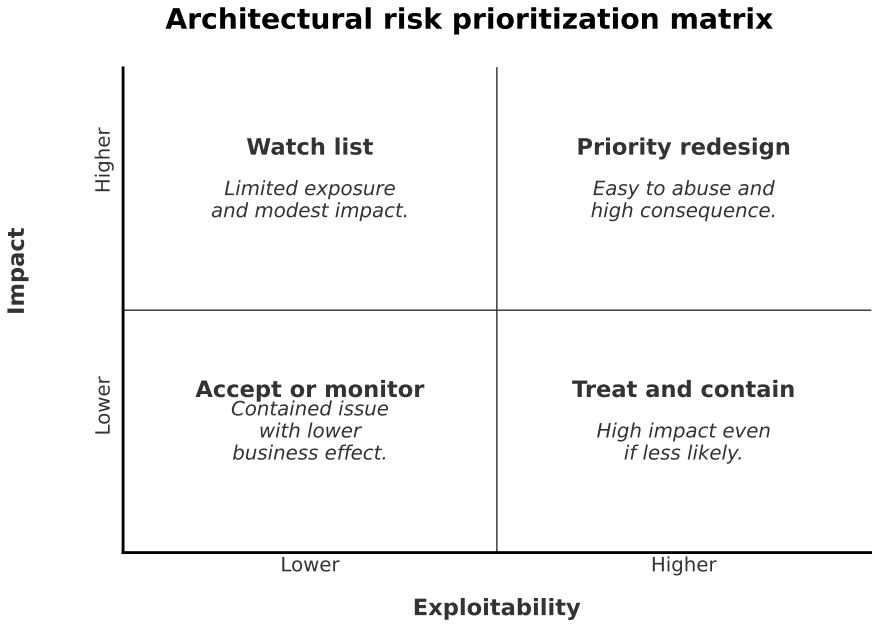

Figure 2.3: Illustrative matrix for architectural risk prioritization.

A mature analysis does not stop at rating risks. It also evaluates whether the architecture supports containment and recovery. This means asking whether the compromise of one component should allow lateral movement, whether audit evidence is sufficient for incident reconstruction, whether secrets and credentials are isolated, and whether critical services fail safely under adverse conditions. These issues are aligned with NIST's systems security engineering guidance, which treats trustworthiness as a function of deliberate engineering choices rather than as the mere absence of known bugs (**Ross et al., 2022**). They also match MITRE's organization of design-level weaknesses, where many serious problems arise from incorrect trust relations, insufficient separation, hidden functionality, improper authorization models, or broader violations of secure design principles (**MITRE, 2025a; 2025b**).

2.1.3 Secure Design Patterns and Anti-Patterns

Once risks and threat paths have been identified, the next step is to shape the architecture through design patterns that consistently enforce security objectives. Secure design patterns are reusable solutions that encode security logic into system structure. They are valuable because they reduce ambiguity, improve consistency across components, and make verification easier in later lifecycle stages. Typical examples include controlled entry through an API gateway, centralized authentication and authorization services, explicit trust contracts between services, segmented data zones, dedicated secret-management mechanisms, and isolated logging pipelines. These structures do not eliminate risk by themselves, but they narrow the attack surface and define clearer enforcement points. OWASP's Secure by Design Framework highlights their value as repeatable architectural solutions (**OWASP Foundation, 2025j**), while CISA's guidance supports the same logic from a secure-by-default product perspective (**Cybersecurity and Infrastructure Security Agency, 2023a**).

Patterns are particularly important in distributed architectures, where security failures often arise not from the absence of controls but from inconsistent enforcement. If one service validates tokens differently from another, if some services access secrets directly while others use a broker, or if authorization is duplicated in incompatible ways, the architecture becomes difficult to reason about and test. Reusable patterns reduce this fragmentation. They also support traceability: when a requirement such as least privilege or controlled disclosure is expressed at design level through a recurring pattern, later implementation and testing activities can verify whether the pattern has been implemented faithfully.

The inverse perspective is equally instructive. Architectural anti-patterns are recurrent structures that systematically generate security weakness. Examples include direct exposure of internal services to external traffic, reliance on shared administrative identities, security checks enforced only on the client side, hard-coded secrets, unrestricted east-west traffic, and implicit trust between components that communicate across boundaries. These anti-patterns are dangerous because they conceal security assumptions inside the architecture itself. Re-

cent OWASP material on insecure design argues that many severe application risks are rooted not in individual coding mistakes but in structural design flaws that late testing cannot fully correct (**OWASP Foundation, 2025b**). OWASP's threat-modeling guidance points in the same direction by encouraging teams to surface those flaws before implementation begins (**2025n**).

The contrast between these two categories captures an essential design principle: secure architecture makes trust explicit and localizes security enforcement, while insecure architecture disperses or obscures it. This distinction matters for the rest of the lifecycle. An implementation team can write correct code and still inherit unnecessary risk from an architecture that assumes excessive trust or weak segmentation.

A technically mature architecture therefore does not aim for absolute elimination of threats; it aims for explicit trust boundaries, verifiable control points, constrained privilege, and resilient failure behavior. This orientation also prepares the ground for the next phase of the lifecycle. Secure implementation is easier when architectural patterns already define where validation occurs, how secrets are handled, which interfaces are privileged, and how dependencies should be isolated. For this reason, security-focused architectural design should be considered the operational foundation of the broader SSDLC rather than a preliminary theoretical exercise.

2.2 Secure Implementation Practices

Implementation is the phase in which security requirements are transformed into executable behavior. At this point, design choices become source code, configuration, build logic, and library usage. This transition is security-critical because even a sound architecture can be weakened by inconsistent validation, unsafe defaults, poor secret handling, or review gaps. Secure implementation should therefore be treated as a controlled engineering activity with two objectives: preserving the intent of earlier requirements and architectural decisions, and preventing common classes of weakness before they propagate into testing or pro-

duction (**Microsoft, 2025b; Souppaya et al., 2022**). Since no single technique is sufficient on its own (**Black et al., 2021**), the present section examines secure implementation through four complementary dimensions: secure coding principles, input validation and data-handling controls, code review, and automated code scanning.

2.2.1 Secure Coding Principles

Secure coding principles define the local rules through which security requirements become repeatable implementation behavior. Their role is not limited to programming style. They reduce ambiguity in security-critical situations and make later review more objective because code can be assessed against explicit expectations rather than vague good practice. This is why organizational coding standards remain important in secure development programs (**Souppaya et al., 2022**). Language-specific guidance, such as the SEI CERT materials, makes those expectations more concrete at implementation level (**Software Engineering Institute, 2020**).

Several principles recur across authoritative sources. One is secure by default: permissive behavior should not be the starting point of an implementation. Another is least privilege, which at code level means scoped tokens, narrowly defined service roles, and restricted access to files, APIs, or database operations. A third is minimization of attack surface: developers should expose only the interfaces and behaviors that are functionally necessary and avoid dormant or overly broad functionality. Finally, trust transitions should be explicit. External input, privileged operations, cryptographic actions, and secret retrieval should never rely on implicit assumptions hidden in scattered helper functions or duplicated logic (**OWASP Foundation, 2025c; 2025h; 2026e**).

These principles are most effective when they are embedded into the development flow. Figure 2.4 shows a practical model in which local checks, peer review, and CI-based security gates reinforce each other. The point is not to prescribe a specific toolchain, but to show that secure implementation is stronger when security checks are distributed across multiple checkpoints instead of being

postponed to the end of development.

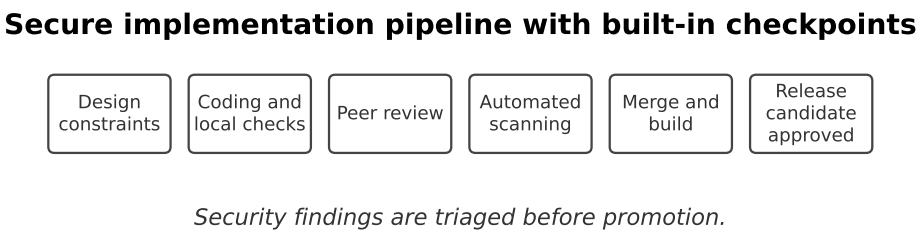

Figure 2.4: Secure implementation pipeline with local checks, peer review, and CI security gates.

2.2.2 Input Validation, Error Handling, and Data Protection

Input validation is one of the clearest examples of how trust must be re-established in code. The objective is not simply to reject malformed data, but to ensure that each input channel accepts only values that are syntactically valid, semantically expected, and safe for downstream use. OWASP recommends allowlist-oriented validation, canonicalization where multiple encodings are possible, and explicit checks for type, length, range, and format (**2025e**). In practice, this means validation should occur on the server side and before the application reaches privileged processing, storage, or sensitive state changes.

Error handling must then ensure that failures do not become disclosure channels. Detailed stack traces, database errors, framework versions, and filesystem details should not be exposed to end users. At the same time, operations must fail in a way that remains observable to maintainers. Secure handling therefore combines non-revealing client responses with detailed server-side logging and correlation data for analysis. OWASP's error-handling guidance addresses the first part of that balance (**2025d**), while its logging guidance addresses the second (**2026c**). Figure 2.5 summarizes this pattern: untrusted input is validated before business logic, while rejected or failed operations produce controlled failures and useful internal evidence rather than implementation detail visible to the client.

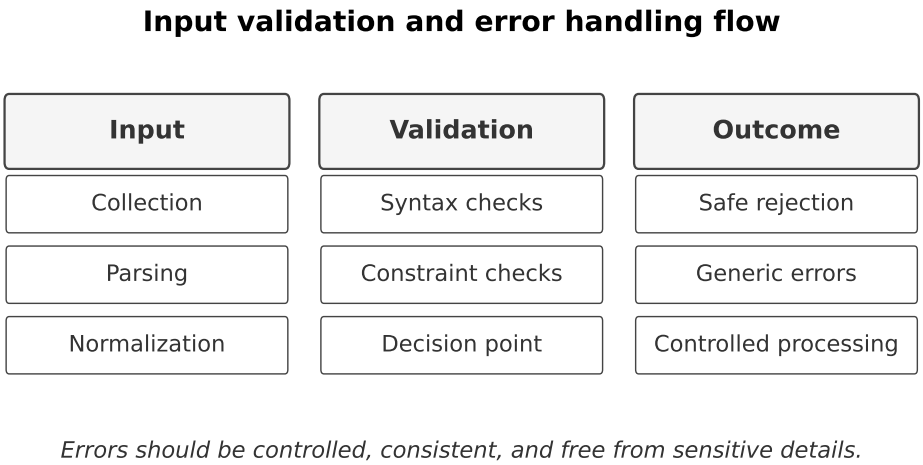

Figure 2.5: Input validation and exception handling flow.

Data protection completes this picture because secure implementation must also preserve control over sensitive information after data has been accepted. Developers should minimize unnecessary collection, restrict access according to server-side authorization, and apply appropriate protection in transit and at rest. OWASP guidance further emphasizes that developers should not improvise cryptographic design, should not store passwords with reversible mechanisms, and should avoid embedding secrets directly in source code or static configuration (**2025c; 2025f; 2025l; 2026e**). In other words, data protection is not a separate operational concern; it is already an implementation responsibility.

2.2.3 Code Review Processes

Security-oriented code review remains essential because it introduces contextual human judgment where automated checks are limited. A reviewer can assess whether the change actually preserves the intent of a requirement, whether authorization is enforced in the right place, whether validation can be bypassed, or whether a small refactoring alters a sensitive data flow. NIST SSDF includes code review among the practices used to verify compliance with secure coding standards (**Souppaya et al., 2022**). Microsoft SDL guidance, and Microsoft's broader secure development and operations material, similarly treat manual re-

view by a separate reviewer as part of disciplined release control (**Microsoft, 2025a; 2025b**).

A useful review process starts with preparation. Reviewers should understand which assets are affected, whether the change touches authentication, authorization, input handling, serialization, secrets, or dependencies, and whether the threat assumptions have changed. OWASP's secure code review guidance is particularly helpful because it recommends reviewing code in context rather than treating security review as a purely line-by-line exercise (**OWASP Foundation, 2025m**). This makes review directly dependent on the quality of earlier lifecycle work: requirements, threat modeling, and architecture all improve implementation review.

Review depth should also vary with risk. Ordinary changes may be handled through focused diff review, while high-risk modules deserve broader baseline analysis. Authentication logic, validation libraries, deserialization paths, cryptographic handling, and other privileged functions generally require more careful scrutiny. Checklists can improve consistency, but only if they support real reasoning instead of replacing it. A review process becomes weak when it is reduced to ceremonial approval without substantive challenge (**2025m**).

2.2.4 Static Analysis and Automated Code Scanning

Automated code scanning is necessary because manual review alone does not scale across large or rapidly changing codebases. Static analysis and related scanning techniques provide repeatability, fast feedback, and a way to enforce minimum security expectations before merge or release. NIST guidance explicitly includes automated code analysis within secure implementation practice, while broader verification guidance recommends combining techniques according to the system's risk profile and development context (**Black et al., 2021; National Institute of Standards and Technology, 2021; Souppaya et al., 2022**).

In practical terms, automated scanning is valuable for findings that are structurally visible in code or adjacent artifacts. This includes unsafe API use, miss-

ing checks around untrusted input, hardcoded secrets, prohibited libraries, and policy violations. Some teams also integrate dependency checks during implementation so that known vulnerable packages are detected early, even if broader third-party risk management is treated elsewhere in the lifecycle (**OWASP Foundation, 2025h**). The main advantage of automation is consistency: the same classes of defect can be checked across many changes without relying entirely on individual reviewer attention.

Automation, however, does not remove the need for human judgment. Static analysis may miss business-logic flaws, over-report low-value findings, or struggle with highly dynamic behavior. Tools therefore need tuning, triage, and explicit ownership. Figure 2.6 illustrates this complementarity. Manual review is generally stronger for logic-heavy authorization questions, while automated scanning is usually more consistent in identifying unsafe APIs, policy violations, committed secrets, and known vulnerable dependencies.

Complementary coverage of review and scanning activities

	Peer review	Static analysis	Dependency scan
Business logic and misuse	High	Moderate	Low
Unsafe APIs and code quality	Moderate	High	Low
Dependency risk	Low	Low	High
Configuration and secrets	Moderate	Moderate	Moderate

Figure 2.6: Illustrative comparison of manual review and automated scanning coverage.

Coding standards define the expected behavior, peer review checks contextual correctness, and automated scanning provides scale and repeatability. Together, they preserve design intent before the software moves into broader verification activities.

2.3 Security Testing Across the Lifecycle

Security testing provides the evidence needed to determine whether security requirements, architectural decisions, and implementation controls are actually effective. In a mature lifecycle, testing is not a final gate but a sequence of verification activities that begins when requirements and threat assumptions are defined and continues as code, configurations, and dependencies evolve (**Black et al., 2021; OWASP Foundation, 2025o; Scarfone, Souppaya, et al., 2008; Souppaya et al., 2022**). Testing must be aligned with lifecycle artifacts: requirements indicate what controls should exist, architecture identifies high-risk trust transitions, and implementation reveals the actual attack surface. The present section examines security testing through three connected dimensions: test planning, technical testing methods, and verification of implemented security controls.

2.3.1 Security Test Planning

Security test planning translates security objectives into a deliberate verification strategy. A useful plan identifies what must be tested, why it must be tested, which technique is appropriate, and what evidence will be accepted as sufficient. NIST SP 800-115 remains particularly valuable on this point because it treats testing as an organized process involving rules of engagement, target identification, technique selection, result analysis, and follow-up mitigation (**Scarfone, Souppaya, et al., 2008**). Although broader than application testing alone, its logic remains directly applicable to software projects: testing should be scoped, repeatable, and tied to explicit objectives rather than performed as an improvised exercise.

In lifecycle terms, good planning begins with traceability. Security requirements, abuse cases, architectural trust boundaries, and prior defect history should all influence the testing strategy. High-risk components usually deserve deeper verification, especially where authentication, authorization, input handling, cryptographic operations, secret use, or tenant separation are involved. OWASP's testing guidance similarly stresses the importance of deriving testing activities

from application structure and risk assumptions instead of applying a uniform checklist blindly (**OWASP Foundation, 2025o**). The implication is that security test planning is not independent from earlier lifecycle work; it depends on how well the system has already been specified and modeled.

A practical plan should also define test environments, preconditions, success criteria, and ownership. Teams need to decide which checks will be executed continuously in development, which require dedicated integration or staging environments, and which findings block release. Figure 2.7 summarizes this logic. The flow begins with requirements, architecture, and threat scenarios; it then maps risks to techniques, identifies test data and environments, and finally feeds results back into remediation and regression testing. This structure is important because unplanned testing often produces findings without decision value: defects are identified, but their relevance, reproducibility, or remediation priority remains unclear.

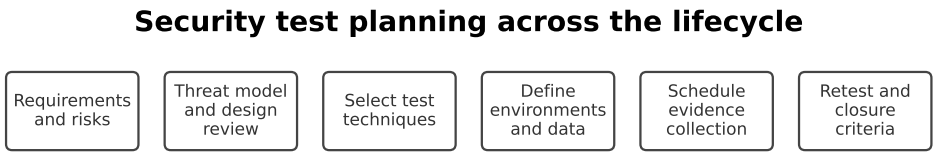

Figure 2.7: Risk-based workflow for security test planning across the lifecycle.

An additional planning issue concerns timing. Some security tests are most effective early, when design or coding errors can still be corrected cheaply; others require a running system and realistic integration behavior. NIST IR 8397 is explicit that organizations should combine automated and human-driven verification methods, including static code scanning, black-box test cases, fuzzing, and web application scanners when applicable (**Black et al., 2021**). This reinforces a key lifecycle principle: the objective is not to choose one preferred testing technique, but to build a sequence of checks that can reveal different classes of weakness at the most useful stage.

2.3.2 SAST, DAST, SCA, and Fuzzing

Security testing across the lifecycle relies on technical diversity because each method reveals a different part of the problem space. Static Application Security Testing (SAST) examines source code or related artifacts without executing the application. Its main value lies in early feedback: developers can identify insecure API use, missing validation, dangerous patterns, or policy violations before code reaches production-like environments. NIST includes static code scanning among the recommended developer verification techniques, and NIST's broader work on source code security analysis highlights its usefulness as a repeatable assurance mechanism during development (**2021; National Institute of Standards and Technology, 2021**).

Dynamic Application Security Testing (DAST), by contrast, evaluates the running application from the outside by interacting with exposed functionality. This makes it particularly useful for observing deployed behavior, including authentication flows, session handling, access control failures, server misconfiguration, and exposure created by runtime integration choices. OWASP's Web Security Testing Guide is especially relevant here because it organizes testing around realistic attack surfaces and manual verification logic rather than assuming that automated crawling alone is sufficient (**OWASP Foundation, 2025o**). DAST is therefore valuable not only as a scanner category, but as a broader runtime testing perspective.

Software Composition Analysis (SCA) addresses a different problem: modern software inherits risk through third-party libraries, transitive dependencies, and package ecosystems. An SCA process attempts to identify included components and correlate them with known vulnerabilities or policy concerns. OWASP Dependency-Check is a representative example of this class of tooling and is designed to detect publicly disclosed vulnerabilities in project dependencies (**2025i**). Even when third-party risk is analyzed more fully in a later chapter, SCA already belongs in the testing discussion because component-related weaknesses can and should be detected during development and pre-release verification.

Fuzzing extends the testing portfolio by exploring how the system behaves

under malformed, unexpected, or randomly generated input. It is particularly effective where robustness and parsing behavior matter, such as protocol handlers, file parsers, APIs, serialization routines, and memory-sensitive code. NIST describes fuzz testing as a method in which invalid or unexpected inputs are supplied and the system is observed for crashes or other undesirable behavior, while modern large-scale platforms such as OSS-Fuzz demonstrate how fuzzing can be integrated continuously for software components that benefit from repeated input exploration (**Google Open Source, 2026; Okun & Fong, 2015**). Fuzzing is therefore best understood as a systematic method for uncovering behaviors that ordinary functional test cases may never reach.

The relationship among these techniques is summarized in Figure 2.8. The matrix does not claim that any method is exclusive to a single phase. Instead, it shows their main strengths. SAST is strongest during implementation and review, DAST becomes more informative once the application is runnable, SCA follows the dependency set throughout development and release preparation, and fuzzing is most useful where interfaces can be exercised repeatedly with generated inputs. Since each technique addresses a different part of the problem space, combining them is essential for adequate coverage of implementation defects, runtime exposure, dependency risk, and unexpected-state behavior.

Where common security testing techniques fit best

	SAST	DAST	SCA	Fuzzing
Developer workflow	High	Low	Moderate	Low
Integration stage	Moderate	Moderate	Moderate	Moderate
Pre-release validation	Moderate	High	High	High
Runtime or staging	Low	High	Moderate	Moderate

Figure 2.8: Indicative mapping of major security testing techniques to lifecycle stages and weakness classes.

2.3.3 Verification of Implemented Security Controls

Finding vulnerabilities is only one part of testing. A disciplined lifecycle also requires verification that the intended security controls have been implemented correctly and are effective under realistic conditions. This is a different question from whether a scanner reports issues. For example, an application may show no obvious injection findings and yet still enforce authorization inconsistently, leak sensitive state through error handling, or omit expected protections around account recovery or session invalidation. Verification must therefore be tied to control intent rather than to tool output alone (**OWASP Foundation, 2025h; Souppaya et al., 2022**).

At this point, a verification standard becomes useful because it structures what should be checked. OWASP ASVS provides a practical basis for this by organizing application security controls into verifiable requirement areas such as authentication, access control, validation, cryptography, error handling, and logging (**OWASP Foundation, 2025h**). In a thesis context, ASVS is useful not as a compliance claim, but as an example of how implemented controls can be tested systematically. The important lifecycle principle is traceability: each relevant control should be linked to a requirement or risk statement, a test method, an observed result, and, if necessary, a remediation and regression action.

This verification activity normally combines positive and negative testing. Positive tests confirm that the intended mechanism works under expected conditions, while negative tests attempt bypass, misuse, or failure-inducing behavior. Both are necessary. An authorization control is not adequately verified by showing that a legitimate user can access an allowed resource; it must also be shown that cross-role or cross-tenant access is rejected. Similarly, input validation is not fully verified by accepting valid data; the system should also demonstrate controlled behavior when inputs are malformed, oversized, or semantically inconsistent (**2025h; 2025o**). Verification is therefore strongest when functional expectations and adversarial expectations are tested together.

Figure 2.9 presents a simple evidence loop for this purpose. Requirements and risks define what should be protected, implemented controls define where

enforcement exists, and test activities generate evidence that supports release decisions or remediation. The loop then closes through regression testing, which is essential because corrected weaknesses can easily reappear after later code changes or dependency updates. This feedback structure is one of the main reasons security testing must be treated as continuous rather than episodic.

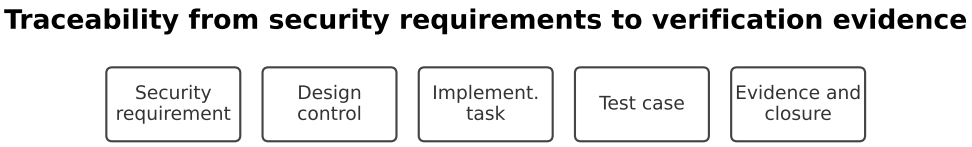

Figure 2.9: Traceability loop for verifying implemented security controls.

When planning, complementary techniques, and control verification are combined, testing becomes the mechanism through which security claims are challenged, supported, and continuously revalidated.

2.4 Configuration and Change Management

Configuration and change management provide the operational discipline that prevents secure design and secure implementation from being undermined by uncontrolled modifications. In software projects, this discipline applies not only to application code, but also to build definitions, dependency manifests, deployment descriptors, secrets handling procedures, infrastructure parameters, and release metadata. NIST's guidance on security-focused configuration management emphasizes that information security should be treated as an integral part of overall configuration management (**Johnson et al., 2019**). More recent pipeline-oriented guidance extends that logic by showing that build, package, and deployment processes themselves have become part of the software security boundary (**Chandramouli et al., 2024a**). For this reason, configuration and change management should be understood as a lifecycle control domain that preserves integrity, traceability, and reproducibility as the system evolves.

From a lifecycle perspective, the key question is not whether change will

happen, but whether change can occur in a controlled and reviewable form. Modern delivery practices accelerate release frequency, yet they also expand the attack surface by concentrating trust in repositories, automation servers, package registries, and deployment pipelines. OWASP's CI/CD guidance explicitly treats these environments as attractive targets because they connect code, credentials, automation, and production delivery in a largely automated flow (**OWASP Foundation, n.d.-a**). A mature approach therefore links change control with three practical concerns: securing build pipelines, controlling dependencies and configuration state, and preserving integrity across version control and release activities.

2.4.1 Secure Build Pipelines

A secure build pipeline is more than a convenience mechanism for compiling and packaging software. It is the place where untrusted code changes are transformed into artifacts that may later be deployed into production. If the pipeline is not adequately protected, an attacker may bypass otherwise strong application controls by tampering with the build environment, injecting malicious dependencies, stealing secrets, or altering outputs after testing has occurred. NIST SP 800-204D highlights this by describing CI/CD pipelines as the flow processes through which source code passes build, test, package, and deploy stages, and by treating security measures within those stages as part of software supply chain assurance (**Chandramouli et al., 2024a**). The security of the build process is therefore inseparable from the security of the resulting software.

In practical terms, secure build pipelines depend on isolation, policy enforcement, and evidence generation. NIST recommends specifying build policies, using secure isolated platforms for builds, hardening build servers, and generating attestation evidence that demonstrates compliance with secure build practices during delivery (**2024a**). OWASP's CI/CD guidance converges on the same idea by stressing isolated build nodes, protected communication with source control, and security checks incorporated directly into the pipeline rather than treated as optional external reviews (**OWASP Foundation, n.d.-a**). This means that the

pipeline itself becomes an enforcement point for organizational rules on what may be built, which tools are trusted, and what conditions must be satisfied before an artifact can progress.

An additional requirement is strict control over identities and secrets used during pipeline execution. Build runners, service accounts, repository tokens, signing keys, and deployment credentials often have privileged access to internal systems. If these identities are overprivileged or if secrets are exposed through logs, environment variables, or repository files, compromise of the pipeline can quickly lead to compromise of the broader delivery process. OWASP's CI/CD guidance therefore recommends denying by default, applying least privilege to pipeline identities, and preventing secrets from being hardcoded in repositories or CI/CD configuration files (**n.d., -a**). The OWASP secrets-management guidance complements this with more specific recommendations on centralized secret handling, encryption, and the use of short-lived credentials where possible (**2026e**). In this context, secure change management includes deciding not only what the pipeline does, but what it is allowed to access while doing it.

Figure 2.10 summarizes these relationships. The diagram shows the build process as a controlled sequence that begins with a reviewed source event and proceeds through an isolated runner, dependency and configuration checks, and a formal policy gate before a release candidate is accepted. The underlying point is methodological: the pipeline should not simply execute changes quickly, but should transform them under controlled conditions and produce evidence about the integrity of that transformation.

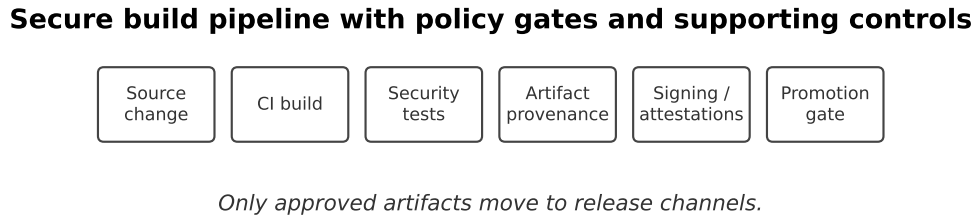

Figure 2.10: Secure build pipeline with policy gates and supporting controls.

This perspective also changes how build failures should be interpreted. A

failed build due to a blocked dependency, a secret leak, a policy violation, or an unsigned change is not merely an operational inconvenience. It is evidence that the change has not yet satisfied the integrity conditions required for trustworthy release. In a secure lifecycle, such failures are part of the control system rather than interruptions of it. Accordingly, secure build pipelines should be designed to produce decisions that are explainable, repeatable, and auditable, not just fast.

2.4.2 Dependency and Configuration Security

Dependency and configuration security address two closely related sources of software risk: inherited functionality that enters the project through external components, and operational behavior that is determined by configuration rather than source code. Both are often underestimated because neither always appears as a classic coding defect. Yet modern applications rely heavily on third-party packages, transitive dependencies, container images, and framework defaults, while operational characteristics are increasingly defined through configuration-as-code, deployment manifests, feature flags, and environment parameters. A secure change management process must therefore control what is included in the system and under which settings it is allowed to run. NIST's configuration-management guidance supports that baseline-control perspective (**Johnson et al., 2019**), while OWASP's material on vulnerable dependency management and outdated components highlights the same issue from the viewpoint of third-party software risk (**OWASP Foundation, n.d.-b; 2025a**).

For dependencies, the first requirement is visibility. OWASP's Top 10 guidance on vulnerable and outdated components states that organizations are likely vulnerable when they do not know the versions of all components they use, including nested dependencies, or when they do not scan regularly for vulnerabilities and apply fixes in a timely manner (**2025a**). The related OWASP cheat sheet on vulnerable dependency management adds that handling third-party components should be treated as a defined process rather than as an ad hoc reaction to scanner output (**n.d., -b**). In practical terms, this means maintaining current manifests and lockfiles, monitoring advisories, evaluating whether updates are

compatible and necessary, and making explicit decisions about remediation, temporary exceptions, or component replacement.

The second requirement is integrity control over dependency resolution. OWASP's CI/CD guidance points out that dependency chain abuse may exploit package resolution behavior through mechanisms such as dependency confusion or typosquatting. As a response, it recommends immutable dependency references, version pinning, integrity verification through hashes or lockfiles, and preference for controlled repositories or private feeds where feasible (**n.d., -a**). These recommendations are important because they shift dependency management from passive inventorying to active prevention of unauthorized or unexpected package substitution. Within a thesis framework, this is a useful example of how change management directly supports security: the approved dependency set becomes part of the system baseline, and deviations from that baseline require deliberate review.

Configuration security follows a similar logic. NIST SP 800-128 treats configuration management as a means of establishing and maintaining system integrity through baselines, controlled changes, and verification of approved states (**Johnson et al., 2019**). In software projects, this principle is especially relevant for environment variables, security headers, logging levels, network exposure, authentication parameters, and deployment descriptors. If these settings are changed informally or are not represented in version-controlled artifacts, teams lose the ability to determine which state was approved, which state was deployed, and whether insecure deviations have emerged over time. Controlled configuration therefore requires approved baselines, reviewable updates, and validation that deployed settings remain aligned with the intended secure state.

Secrets should be handled as a special class of configuration item because they are both highly sensitive and operationally dynamic. The OWASP Secrets Management Cheat Sheet emphasizes centralized storage, controlled provisioning, auditing, rotation, and measures that prevent secrets from leaking into source repositories, binaries, logs, or shell history (**OWASP Foundation, 2026e**). This is particularly relevant in CI/CD and deployment scenarios, where secrets are

frequently injected automatically into build or runtime environments. A secure configuration process should therefore distinguish between ordinary operational parameters and confidential values that require stronger access control, shorter lifetimes, and explicit rotation procedures.

Figure 2.11 illustrates the resulting control loop. A baseline is established, a change request is evaluated, modifications are implemented in reviewable configuration artifacts, validation and scanning are performed, and the approved baseline is updated only after controlled deployment. Drift detection and exception handling are shown as continuous activities because configuration security is not established permanently; it must be maintained as systems and dependencies continue to change.

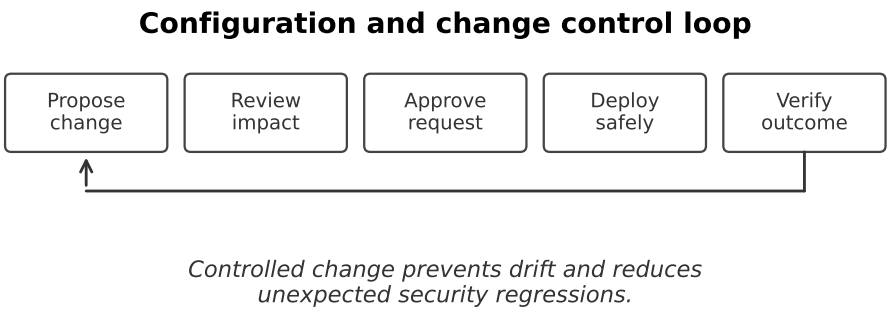

Figure 2.11: Configuration baseline and change control loop for secure software delivery.

2.4.3 Release and Version Control Considerations

Version control systems and release procedures are often discussed as developer productivity mechanisms, but from a security standpoint they are also evidence systems. They record who introduced a change, when it was reviewed, how it entered an approved branch, which files were included in the release, and what integrity information accompanies the resulting artifact. This traceability matters because secure release decisions depend on more than successful compilation. They depend on whether the provenance of a change is understandable and

whether the produced artifact can be linked back to approved inputs and controlled processes (**Chandramouli et al., 2024a; OWASP Foundation, n.d.-a**).

OWASP's CI/CD guidance is particularly explicit about secure source control configuration. It recommends avoiding auto-merge rules for sensitive pipelines, requiring pull request review that cannot be bypassed, leveraging protected branches, requiring signed commits, enabling multifactor authentication, and managing repository permissions carefully (**n.d., -a**). These controls reduce the chance that unauthorized, insufficiently reviewed, or ambiguously attributable changes become part of the software baseline. They also reinforce the principle that code repositories should not merely store files, but preserve accountable history.

NIST SP 800-204D extends this logic into deployment and GitOps-style operations. It states that changes should not be applied manually at runtime; instead, changes should be made to the relevant code and a new release should be triggered, so that the authoritative record remains in version-controlled commits (**Chandramouli et al., 2024a**). This is a valuable lifecycle principle because it minimizes untracked divergence between what teams believe is deployed and what is actually running. Secure change management is therefore strengthened when operational changes are routed back through reviewed repository history instead of being introduced through manual interventions on live systems.

Release integrity then depends on the evidence attached to the artifact itself. The SLSA framework describes software supply chain controls intended to prevent tampering, improve integrity, and secure packages and infrastructure (**SLSA, 2025**). The Update Framework, by contrast, focuses specifically on securing software update systems, including against attacks involving compromised repositories or signing keys (**TUF Authors, 2026**). These frameworks represent what trustworthy release practice aims to achieve: signed artifacts, verifiable provenance, integrity metadata, and update mechanisms that resist substitution or rollback attacks.

Figure 2.12 brings these ideas together. Repository history, reviewed and signed changes, artifact provenance, archived release evidence, and deployment or rollback decisions are represented as one integrity chain rather than separate

administrative tasks. This representation matters because secure release is not a final ceremony at the end of development. It is the culmination of earlier controls on repository governance, dependency approval, build integrity, and configuration management. When those controls are weak, release assurance becomes largely symbolic.

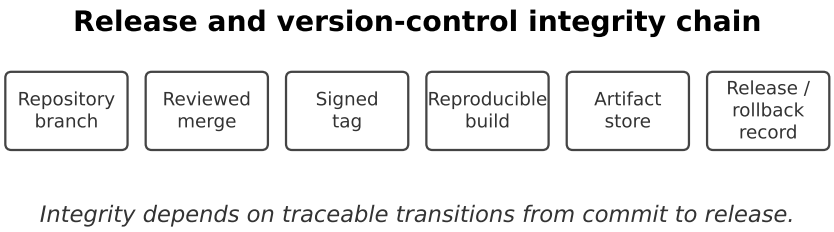

Figure 2.12: Release and version-control integrity chain from reviewed change to deployment decision.

When secure build pipelines, dependency governance, configuration control, and release integrity are integrated, change becomes traceable and governable—precisely the condition required for deployment, operations, and long-term maintenance.

Chapter 3

Deployment, Operational Security, and Software Supply Chain Integrity

3.1 Secure Deployment and Environment Hardening

Secure deployment is the lifecycle stage in which a validated software artifact is transformed into an operational service. From a security perspective, this transition is not merely a release activity but a trust-preserving process in which software, configuration, identities, and runtime permissions must remain consistent with the assumptions established during design, implementation, and verification. If deployment is handled as an administrative routine rather than as a controlled security process, previously mitigated risks may be reintroduced through misconfiguration, excessive privilege, weak separation between environments, or unsafe handling of secrets. For this reason, deployment hardening should be considered a continuation of secure development rather than a separate operational concern. NIST SSDF frames these controls as part of a broader secure software practice baseline (**Souppaya et al., 2022**), while NIST's DevSecOps guidance shows how they extend into the automated delivery path itself (**Chandramouli,**

2022).

A first requirement is the clear separation of environments used across the lifecycle. Development, build, staging, testing, and production should not be treated as interchangeable spaces, because each environment exposes different assets, trust assumptions, and attack opportunities. The Secure Software Development Framework explicitly notes that organizations should enumerate and protect their environments to reduce the risk of lateral movement between them (**Souppaya et al., 2022**). In practice, this means controlling promotion paths, restricting administrative access, separating credentials, and ensuring that only approved artifacts and configuration bundles can cross into production. The deployment process should therefore be implemented as a sequence of gated transitions rather than as a direct copy of code and configuration from one environment to another.

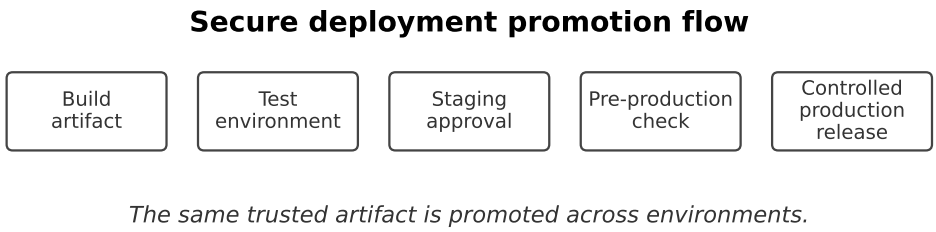

Figure 3.1: Secure deployment as controlled promotion across isolated environments.

Baseline hardening of deployment targets

Hardening begins with the deployment target itself, whether the target is a traditional server, a virtual machine, a container host, or a managed orchestration platform. A useful baseline principle is minimization: only required services, interfaces, packages, and administrative paths should be enabled. NIST guidance on general server security recommends dedicating systems to the intended service where possible, identifying all network services to be exposed, restricting privileged activities, and disabling unnecessary functionality that expands the attack surface (**Scarfone, Jansen, & Tracy, 2008**). Although this guidance predates cloud-native platforms, the underlying logic remains valid: the

less functionality a deployment target exposes, the fewer opportunities exist for exploitation and post-compromise movement.

Baseline hardening also requires that configuration be repeatable and reviewable. In modern delivery pipelines, this objective is commonly achieved through infrastructure as code and policy as code, which allow deployment settings to be versioned, tested, and promoted under change control. NIST SP 800-204C treats infrastructure as code, policy as code, and observability as code as distinct yet interrelated parts of a cloud-native DevSecOps environment (**Chandramouli, 2022**). This is significant because it shifts hardening from manual server preparation to declarative controls that can be reviewed before release. A hardened environment is therefore not simply a well-administered runtime; it is an environment whose security assumptions are encoded, reproducible, and continuously validated.

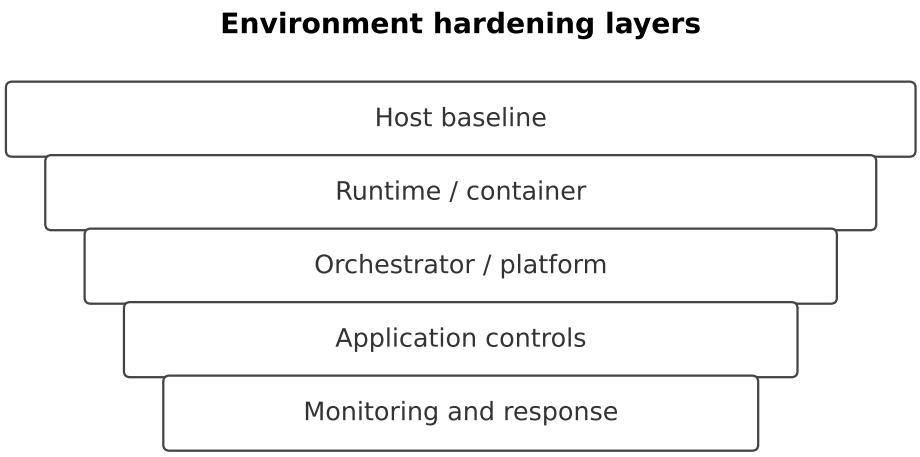

Figure 3.2: Layered view of environment hardening controls for modern deployment targets.

At the host and platform level, hardening measures typically include timely patching, service minimization, secure remote administration, logging protection, and strong identity separation between operators and workloads (**Scarfone, Jansen, & Tracy, 2008**). In containerized deployments, equivalent concerns appear at additional layers. NIST's container security guidance emphasizes that risks exist not only in the container image, but also in the host operating system,

registry, orchestrator, and runtime configuration (**Souppaya et al., 2017**). As a result, deployment hardening should avoid focusing narrowly on the application package alone. A secure deployment posture depends on the integrity of the lower platform layers, because weaknesses in the host or orchestration plane can invalidate protections implemented inside the application.

Hardening cloud-native runtime environments

Cloud-native deployment targets introduce important hardening requirements that differ from those of monolithic server deployments. Containerized workloads should be packaged with minimal base images, pinned dependencies, explicit runtime users, and a default expectation of least privilege. OWASP guidance recommends avoiding privileged containers, limiting Linux capabilities, constraining inter-container communication, and preferring read-only file systems where feasible (**OWASP Foundation, 2026b**). These recommendations align with NIST SP 800-190, which treats image construction, orchestrator settings, and runtime isolation as interdependent parts of container security rather than as independent checks (**Souppaya et al., 2017**). Hardening, in this sense, is achieved not by a single control but by the combined effect of reduced privileges, reduced attack surface, and better verification of what is actually running.

In orchestration platforms such as Kubernetes, deployment hardening must also address the control plane and workload governance model. Official Kubernetes security guidance and the NSA/CISA hardening guide both stress the importance of restricting access to the API server, using role-based access control, protecting kubelet interfaces, applying network policies, and enforcing workload restrictions through admission control and pod security mechanisms (**Kubernetes Authors, 2025; National Security Agency & Cybersecurity and Infrastructure Security Agency, 2022**). Kubernetes also provides a built-in Pod Security Admission controller that can enforce predefined policy levels such as `baseline` and `restricted` at namespace level (**Kubernetes Authors, 2024**). From a lifecycle perspective, these controls are valuable because they prevent insecure deployment manifests from reaching runtime, thereby converting hardening expec-

tations into enforceable platform policy.

Secrets handling is another central part of deployment hardening. Credentials, tokens, certificates, and API keys are frequently exposed not through flaws in cryptographic algorithms but through weak deployment practices, such as embedding secrets in images, reusing long-lived credentials, or distributing secrets through uncontrolled configuration channels. OWASP recommends centralized storage, controlled provisioning, auditing, and rotation of secrets, while Kubernetes guidance warns that environment-variable-based distribution may be more prone to leakage in some scenarios (**2025; OWASP Foundation, 2026e**). Accordingly, a hardened deployment should prefer external secret management mechanisms, scoped access policies, and short-lived identities over static credentials bundled with the application artifact.

Deployment hardening control matrix

Phase	Host	Runtime	Platform	Application
Patch level	Baseline	Minimal	Hardened	Verified
Identity	Service account	Scoped	RBAC	Least privilege
Network policy	Firewall	Namespace	Segmentation	Ingress rules
Secrets	Store	Inject	Rotate	Never hardcode
Logging	System logs	Runtime logs	Audit trail	App events

Figure 3.3: Examples of deployment and hardening controls by operational layer.

A further requirement concerns deployment integrity and accountability. Even when the artifact itself has passed earlier verification, organizations still need confidence that the exact approved artifact is the one promoted to production, with the intended configuration and without unauthorized substitution. For this reason, secure deployment benefits from digest pinning, signed artifacts, provenance-aware release procedures, and event trails that record who approved, changed, and deployed a release. NIST's DevSecOps guidance is especially relevant here because it ties provenance and promotion controls to the delivery path (**Chandramouli, 2022**), while SSDF reinforces the broader expectation of traceable and protected

software delivery (**Souppaya et al., 2022**). This form of release integrity is especially relevant in environments where the same pipeline manages frequent updates, because deployment speed can otherwise weaken assurance if approval and traceability controls are informal.

Overall, secure deployment and environment hardening should be understood as a layered discipline that preserves assurance from pre-release activities into runtime operation. Effective hardening combines environment separation, platform minimization, declarative configuration, runtime least privilege, orchestrator policy enforcement, disciplined secrets management, and traceable release promotion. When these controls are consistently applied, deployment becomes a managed extension of the secure software lifecycle rather than the point at which security assumptions begin to erode.

3.2 Operational Security Controls

Once software has been deployed, security depends not only on the correctness of the released artifact but also on the quality of the operational controls that observe, protect, and sustain the service over time. At this stage, the security objective shifts from preventing insecure code from entering production to preserving trustworthy behavior under changing conditions. New vulnerabilities are disclosed, configurations drift, attack techniques evolve, and systems are exposed to misuse patterns that may not have been visible during pre-release testing. Operational security controls are therefore a continuation of lifecycle security: they provide the monitoring, remediation, and response capabilities needed to detect deviations from expected behavior and to restore control when those deviations become security events (**National Institute of Standards and Technology, 2011; Souppaya et al., 2022**).

Operational controls can be grouped into three interdependent capabilities: visibility (monitoring, logging, auditing), correction (patching, vulnerability management), and coordinated response (incident handling). These reinforce one another: monitoring without remediation creates backlogs, patching without visi-

bility creates blind trust in incomplete inventories, and incident response without reliable logs becomes slow and error-prone (**National Institute of Standards and Technology, 2006; 2022b; 2025**).

Monitoring, logging, and auditing

Monitoring provides the continuous awareness needed to recognize when a system is drifting away from its expected security posture. NIST describes information security continuous monitoring as maintaining ongoing awareness of threats, vulnerabilities, and security controls to support risk-based decisions (**2011**). In the context of software operations, this means collecting evidence not only from infrastructure components but also from the application layer itself. Authentication events, authorization failures, configuration changes, privilege use, administrative actions, service startup conditions, API abuse indicators, and integrity-relevant deployment actions should all be considered part of the monitoring surface. Security monitoring is most useful when it reflects the architecture and trust assumptions of the deployed service, rather than merely collecting generic host or network telemetry.

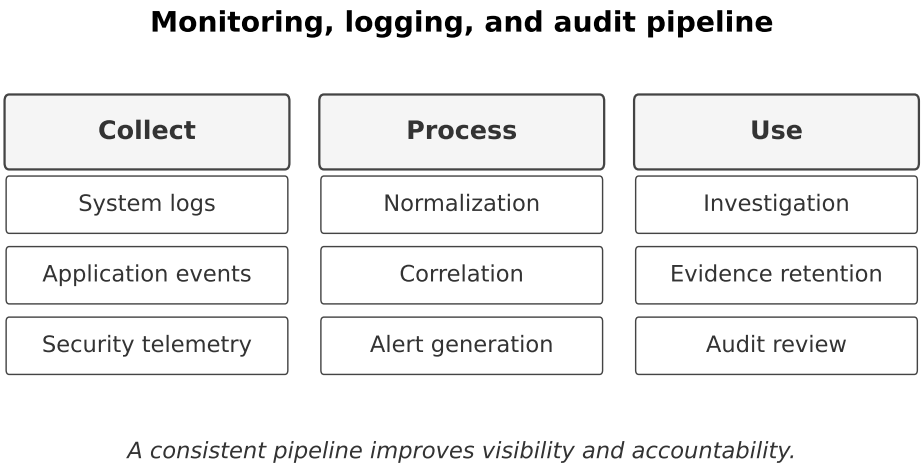

Figure 3.4: Operational monitoring, logging, and auditing pipeline from telemetry generation to triage, alerting, and audit use.

Effective logging is a design decision as much as an operational one. NIST

log management guidance emphasizes the need for consistent generation, transmission, storage, analysis, and disposal of log data across the enterprise (**2006**). OWASP guidance complements this view by stressing that application logs should capture security-relevant events with sufficient context for investigation, while avoiding the disclosure of sensitive data or the creation of new attack surfaces through insecure logging practices (**OWASP Foundation, 2026c**). The key implication is that application logging cannot be postponed until late operations. Logging requirements should be anticipated during implementation and verified during testing, so that production systems produce evidence that is both actionable and forensically useful.

Auditing extends monitoring by supporting accountability and retrospective verification. Not every event requires a real-time alert, but security-significant actions should remain traceable to a principal, a timestamp, a target, and an outcome. This requirement is especially important for administrative interfaces, privileged actions, configuration changes, access to sensitive data, and deployment events. In practice, auditability depends on several technical disciplines working together: time synchronization, protected log transport, retention policies, normalized event structures, and clear separation between operational logs and audit records. If logging is incomplete, inconsistent, or modifiable by the same actor being monitored, the organization loses both detection capability and evidentiary value.

Patch management and vulnerability remediation

Operational security also requires a disciplined method for correcting weaknesses after deployment. Vulnerabilities discovered in application code, third-party components, operating systems, middleware, containers, and orchestration platforms do not all carry the same urgency. NIST SP 800-40 Rev. 4 frames enterprise patch management as a planning and governance problem, not merely a tool problem, and defines patch management as the process of identifying, prioritizing, acquiring, installing, and verifying patches, updates, and upgrades across the organization (**National Institute of Standards and Technology, 2022b**).

This broader perspective is essential because timely patching depends on asset knowledge, ownership, maintenance windows, rollback planning, dependency awareness, and business constraints.

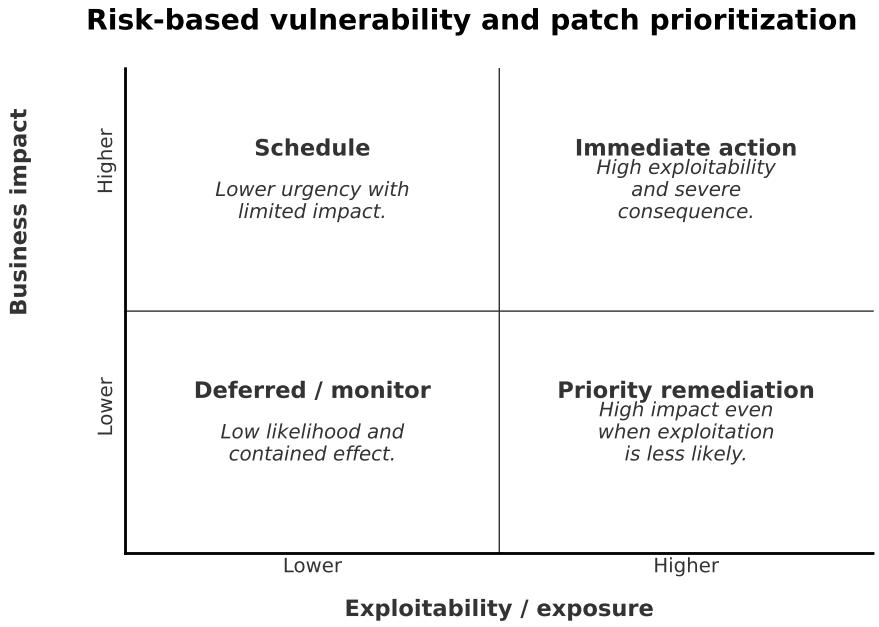

Figure 3.5: Risk-based prioritization loop for vulnerability remediation in operations.

A mature remediation process is risk-based rather than purely severity-based. Technical severity metrics should be interpreted together with asset criticality, exploitability, exposure, and evidence of active exploitation. CISA's Known Exploited Vulnerabilities Catalog helps prioritize remediation beyond generic CVSS rankings (**Cybersecurity and Infrastructure Security Agency, 2026b**), while OWASP's vulnerability management guidance frames remediation as a continuous cycle rather than an occasional scanning activity (**OWASP Foundation, 2020**). Post-deployment remediation should feed back into the development process by revealing fragile components, recurring dependency risks, and security debt. The specific mechanics of vulnerability prioritization, update assurance, and verification are examined further in the maintenance discussion that follows.

Incident response considerations

Even with strong preventive and detective controls, some security events will become incidents. Operational security must therefore include a response capability that is technically informed, well-rehearsed, and integrated with the rest of the lifecycle. NIST SP 800-61 Rev. 3 positions incident response as part of broader cybersecurity risk management and aligns it with the functions of the Cybersecurity Framework 2.0 rather than treating it as a standalone emergency activity (**National Institute of Standards and Technology, 2025**). This is an important shift for software-centric systems: incident response is not only about crisis handling after compromise, but also about preparing systems, teams, and information flows so that detection, containment, eradication, and recovery can occur without unnecessary delay.

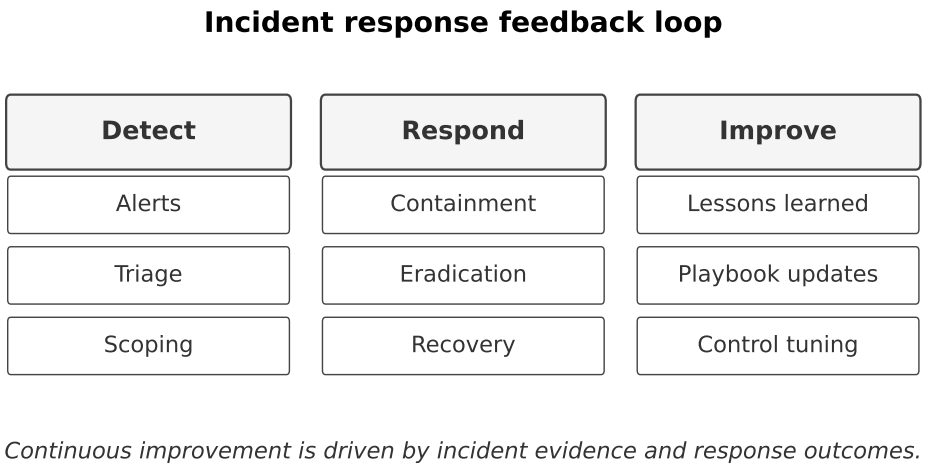

Figure 3.6: Operational incident response loop with feedback to engineering, deployment, and governance activities.

For software operations, incident response considerations begin before any incident occurs. Teams need current asset inventories, ownership maps, contact paths, escalation thresholds, evidence-preservation procedures, and predefined technical options for isolation and rollback. CISA's incident and vulnerability response playbooks illustrate the value of standardized decision paths, explicit roles, and documented communication steps during security events (**Cybersecurity**

and Infrastructure Security Agency, 2024a). At the application level, this preparation also includes the ability to disable risky features, rotate secrets, revoke tokens, redeploy fixed versions, and collect volatile evidence from logs, metrics, traces, and runtime platforms. If these actions require improvisation under stress, containment typically becomes slower and more disruptive.

A further requirement is that post-incident activity should produce engineering knowledge, not just administrative closure. Security incidents often reveal weaknesses that cut across multiple lifecycle stages: insufficient telemetry, weak trust boundaries, over-permissive runtime identities, unclear ownership of dependencies, or slow remediation workflows. The lessons learned from incident handling should therefore be translated into design changes, implementation safeguards, test cases, hardening rules, and updated operational procedures (**National Institute of Standards and Technology, 2025; Souppaya et al., 2022**). In this way, incident response becomes part of the continuous improvement loop of the secure software lifecycle. Operational controls are most effective when they do more than react to failures; they should also reduce the likelihood that the same class of failure will recur in future releases.

3.3 Software Supply Chain and Third-Party Component Security

Modern software is rarely built from code written entirely in-house. Production systems typically combine first-party code with open-source libraries, package repositories, container base images, software development kits, hosted APIs, managed services, and update channels operated by external parties. As a result, software assurance can no longer be limited to the correctness of internal implementation alone. Security must also account for the origin, integrity, maintenance, and operational behavior of the components and services that become part of the delivered system. From a lifecycle perspective, the attack surface extends beyond the application boundary to include the entire chain through which code, dependencies, build outputs, and updates are selected, assembled, and con-

sumed (**Chandramouli et al., 2024b; National Institute of Standards and Technology, 2022a; Souppaya et al., 2022**).

Organizations often depend on components that they did not design, cannot fully audit, and do not control operationally. A vulnerable or malicious package can introduce compromise before code reaches production, and a poorly governed update mechanism can replace trusted functionality with tampered content after release. Software supply-chain security should therefore be treated as a continuous risk-management activity spanning selection, ingestion, build, release, deployment, and maintenance (**National Institute of Standards and Technology, 2022a; Souppaya et al., 2022**).

Dependency evaluation

A disciplined dependency evaluation process begins with the recognition that not all external components carry the same level of risk. Critical runtime libraries, identity-related SDKs, cryptographic modules, package managers, container base images, and infrastructure operators can have security implications far greater than their visible size in the application. Evaluation should therefore consider not only functionality, but also component criticality, transitive dependency depth, maintainer activity, source provenance, release discipline, vulnerability history, licensing constraints, and the operational consequences of failure or compromise. NIST frames supply-chain risk management as an organizational activity integrated with broader risk management, not merely as a technical scan of package versions (**National Institute of Standards and Technology, 2022a**). In practice, this means that engineering, security, legal, and procurement functions may all have a role in deciding whether a dependency is acceptable for a given system context.

Automated software composition analysis is a useful baseline, but it is not sufficient as the sole acceptance mechanism. Tools such as OWASP Dependency-Check can identify known vulnerabilities associated with project dependencies, yet known CVEs represent only one part of supply-chain risk (**OWASP Foundation, 2025i**). A component may have no currently disclosed vulnerability

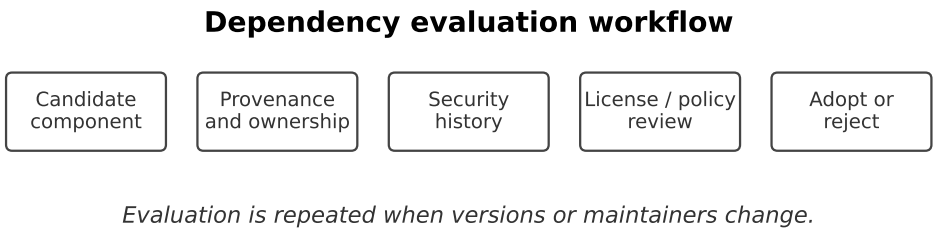

Figure 3.7: Structured workflow for evaluating third-party software components before adoption.

and still remain high-risk because it is effectively unmaintained, depends on untrusted mirrors, uses insecure update practices, or is governed by a project with weak review and release controls. This is why dependency evaluation benefits from combining several perspectives: vulnerability identification, maintainership signals, release integrity, and evidence that secure development practices are actually followed.

Two particularly useful frameworks support this broader evaluation. OWASP SCVS provides a structured way to assess activities and controls that reduce software supply-chain risk (**2025k**). OpenSSF Scorecard complements that view by offering automated checks for project practices such as branch protection, code review, dependency pinning, signed releases, continuous testing, and fuzzing, thereby helping consumers compare packages using more than popularity or convenience (**Open Source Security Foundation, 2025**). These mechanisms should not be treated as absolute trust scores, but as decision support. Their real value lies in highlighting where additional review, compensating controls, or local isolation measures are needed before a component is introduced into the baseline.

A further consideration is that dependency approval should be conditional, not binary. Some components may be acceptable only when pinned to approved versions, isolated within specific runtime boundaries, denied direct access to secrets, or monitored with tighter update and remediation rules. This is especially relevant for dependencies that provide network-facing, identity-related, or data-processing functions. In this sense, dependency evaluation is best understood

as an architectural governance activity: it determines how much trust is being granted to external software and under which constraints that trust remains acceptable.

Software Bill of Materials

Once third-party components are selected, organizations need a reliable method to record what has actually been included in a build and release. The Software Bill of Materials (SBOM) addresses this requirement by providing a formal record of component identities and their dependency relationships. NTIA defines an SBOM as a formal record containing the details and supply-chain relationships of various components used in building software (**National Telecommunications and Information Administration, 2021**). The importance of this artifact is practical rather than purely descriptive: it enables faster vulnerability triage, more precise exposure analysis, more transparent supplier communication, and more consistent remediation when a vulnerable component is disclosed.

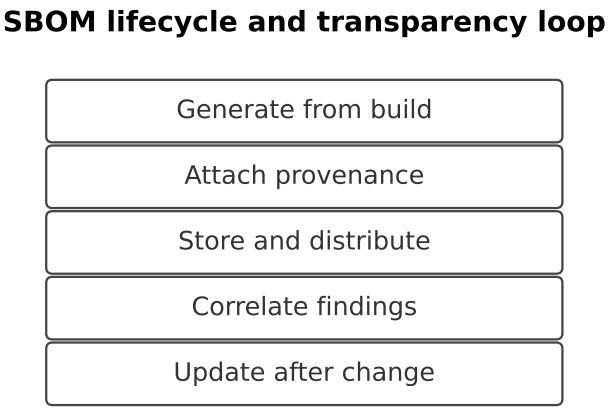

Figure 3.8: SBOM generation, publication, deployment, and reuse across release and operational activities.

An SBOM is most useful when it is generated close to the build process, updated as dependencies change, and attached to the released artifact in a way that preserves traceability. If the SBOM is created manually, generated from incomplete inventories, or allowed to drift away from the actual release contents,

its operational value declines rapidly. The objective is not merely to produce a document for compliance, but to maintain a machine-consumable representation of the software composition that can support downstream decisions during deployment and operations. This includes linking the SBOM to version metadata, release identifiers, vulnerability feeds, and remediation workflows so that when a dependency issue emerges, teams can determine quickly whether a given application version is affected.

SBOM quality also matters. Minimum data fields are necessary, but completeness, freshness, component naming consistency, and the treatment of transitive dependencies all affect whether the artifact can be used effectively. In modern delivery environments, the most mature model is to treat the SBOM as a living release artifact rather than as a one-time procurement attachment. It should be regenerated when the build inputs change, redistributed when releases are repackaged, and checked during intake by downstream consumers. This aligns well with the SSDF view that external software components require ongoing governance throughout the lifecycle (**Souppaya et al., 2022**).

Risks in third-party integrations

Supply-chain exposure is not limited to packages embedded in a build. Third-party integrations introduce risk even when the external functionality remains outside the application artifact itself. API-based services, analytics SDKs, payment gateways, identity providers, telemetry agents, container registries, package mirrors, and automatic update channels all create trust relationships that influence the security posture of the system. These relationships may expand the scope of sensitive data processing, introduce privileged tokens, create hard availability dependencies, or allow remote changes in behavior without corresponding changes in the local code repository.

For this reason, third-party integration review should examine more than interface correctness. Relevant questions include the provider's security update model, authentication and authorization requirements, data residency implications, logging and auditability, failure handling behavior, support for version

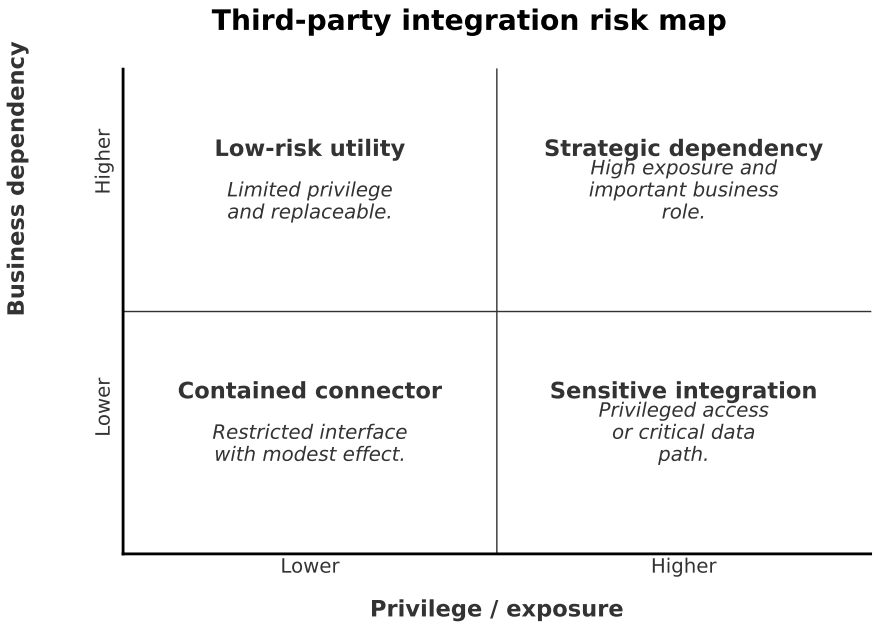

Figure 3.9: Representative third-party integration types, associated risks, and lifecycle controls.

pinning, and the degree to which local teams can disable or constrain the integration if conditions change. CISA's software acquisition and secure-by-demand guidance is useful in this area because it shifts attention toward evidence that suppliers can provide about their development, release, vulnerability handling, and support practices, rather than relying on generic claims of security maturity (**Cybersecurity and Infrastructure Security Agency, 2024b; 2024c**).

A major control objective for third-party integrations is integrity of origin and update. SLSA describes progressive assurance measures for sources, builds, and dependencies, offering a common language for strengthening trust in software artifacts over time (**SLSA, 2025**). Sigstore adds practical signing and verification support for artifacts such as release files, container images, binaries, and SBOMs, while recording signing events in a tamper-resistant public log (**Sigstore Project, 2025**). The in-toto framework extends this logic by making the performed supply- chain steps transparent and verifiable from initiation to end-user installation (**in-toto Authors, 2025**). For software update channels specifically, TUF provides a framework for protecting update systems even when

repositories or signing keys are compromised (**TUF Authors, 2026**). Taken together, these mechanisms do not eliminate supplier risk, but they substantially improve the ability to verify provenance, detect tampering, and resist unsafe update flows.

At the architectural level, integration risk should be reduced through technical containment. Useful controls include least-privilege scopes for third-party identities, network segmentation, explicit egress controls, sandboxing of plug-ins or agents, strict timeout and retry policies, secret rotation, release canaries, and kill-switch mechanisms for disabling problematic functionality. These measures are particularly important when a supplier compromise or external service outage would otherwise propagate immediately into the primary application environment. Third-party trust should therefore be bounded by design rather than assumed as a default property.

3.4 Secure Maintenance and Continuous Improvement

Software security does not end when an application is deployed. Maintenance is the phase in which organizations either preserve earlier security investments or quietly lose them. Configuration shortcuts, delayed patching, unsupported components, rushed hotfixes, and weak feedback loops can gradually erode trustworthiness even when initial design and implementation were strong. Secure maintenance should therefore be treated as a first-class lifecycle activity with its own governance, verification expectations, and evidence trail (**Cybersecurity and Infrastructure Security Agency, 2023a; National Institute of Standards and Technology, 2022b; Souppaya et al., 2022**).

A mature maintenance model must track supported versions, control how updates are prepared and released, verify that fixes have been correctly deployed, and ensure that recurring weaknesses influence future engineering practice. Not every security issue can be resolved immediately through a clean code change; some require temporary mitigations, compensating controls, staged rollout, or

coordinated communication (**National Institute of Standards and Technology, 2022b; 2025; OWASP Foundation, 2020**).

Maintenance as a continuing security obligation

Deployed software remains part of the attack surface for as long as it is supported and reachable. New vulnerabilities may emerge in first-party code, bundled components, container images, runtimes, or external services after release. The objective of maintenance is therefore to preserve acceptable risk over time through controlled change in which security, availability, and traceability remain balanced.

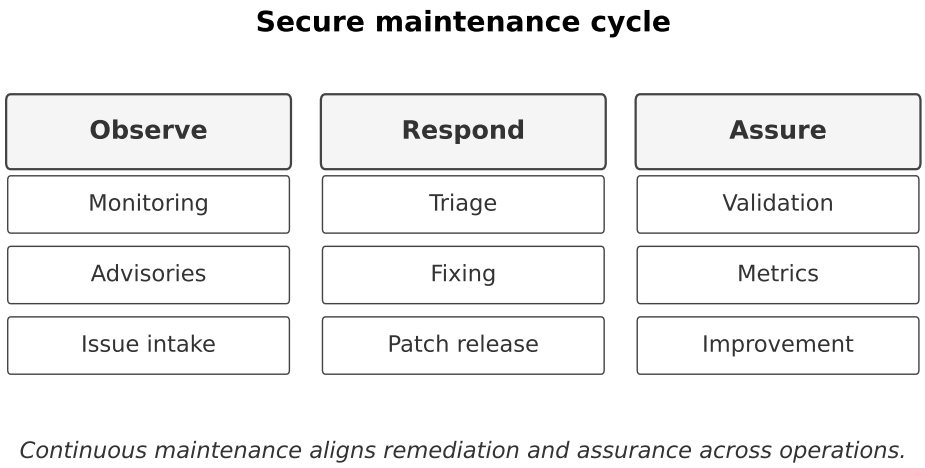

Figure 3.10: Secure maintenance as a controlled cycle connecting detection, prioritization, implementation, verification, monitoring, and improvement.

From a governance perspective, maintenance should preserve a clear relationship between the running system and the approved software baseline. This includes version inventories, support status, release history, known exceptions, and evidence that deployed changes were authorized and verified. Maintenance work should remain tied to source control, ticketing, testing records, and release metadata so that an organization can later determine what changed, why it changed, when it changed, and which systems received the update (**Souppaya et al., 2022**).

A secure maintenance strategy also depends on defined support policies. If products or internal services remain in use beyond their supported versions, teams may inherit unpatched vulnerabilities without a credible path to vendor fixes or internal remediation. Conversely, if updates are pushed without adequate testing, organizations may introduce regressions that weaken both security and service reliability. Mature maintenance therefore combines predictable servicing windows, clear severity thresholds for out-of-band fixes, rollback preparation, and communication paths for stakeholders affected by the change. These ideas align closely with secure-by-design expectations: security improvements should be part of the product's normal support model rather than exceptional activities performed only after public pressure or active exploitation appears (**Cybersecurity and Infrastructure Security Agency, 2023a**).

Another important point is that maintenance should be evidence-based. Operational logs, monitoring alerts, asset inventories, vulnerability feeds, SBOM updates, supplier advisories, and customer reports all provide signals about where security work is needed. If those signals are fragmented or not connected to release management, organizations are forced into reactive maintenance. If they are integrated, the maintenance process becomes more deliberate and defensible. The result is not perfect prevention, but a more stable ability to detect, prioritize, and respond to change before small weaknesses become recurring incidents.

Vulnerability remediation and update assurance

The most visible maintenance activity is vulnerability remediation, but effective remediation requires more than a queue of scanner findings. Organizations need a structured intake and triage process that combines automated detection with human judgment. Relevant inputs may include vulnerability scanners, incident observations, coordinated disclosures from researchers, supplier bulletins, dependency intelligence, and exploit-focused sources such as the CISA Known Exploited Vulnerabilities Catalog (**2026a; 2026b; OWASP Foundation, 2020**). Each input should be validated against affected versions and real deployment context before remediation work is planned, because false positives, duplicate

findings, and de-scoped assets can otherwise distort prioritization and waste limited engineering effort.

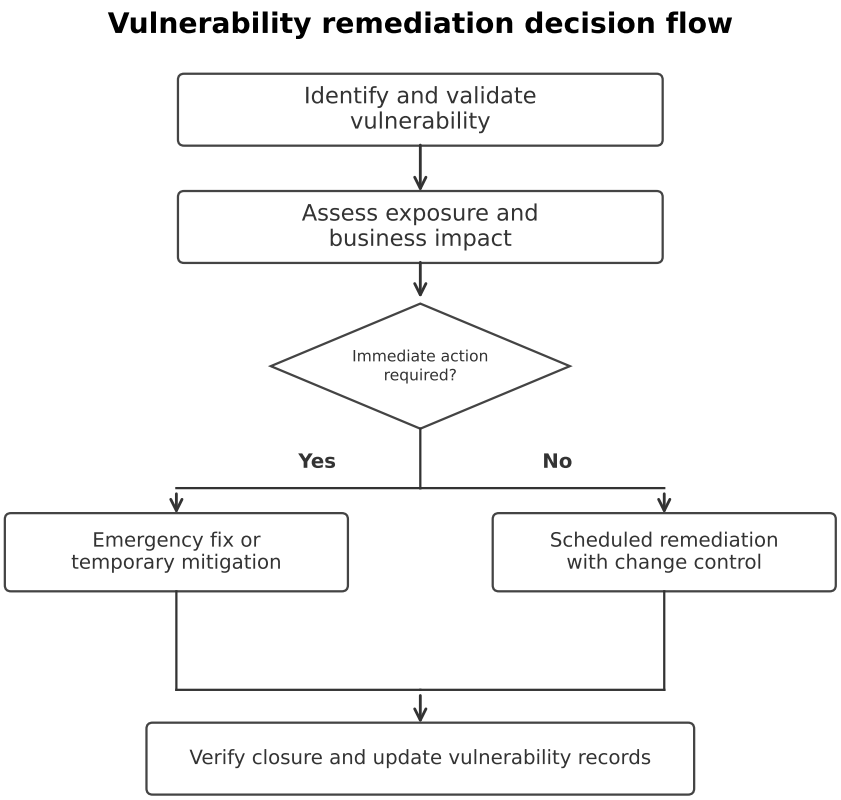

Figure 3.11: Decision flow for risk-based vulnerability maintenance, from intake and triage to rollout, closure, and feedback.

Prioritization should account for more than CVSS-like severity labels. Exploit status, asset criticality, exposure to untrusted networks, data sensitivity, availability constraints, compensating controls, and the age of the affected finding all influence urgency. NIST's patch management guidance emphasizes that patching is a risk-management activity requiring enterprise strategy and prioritization, not a mechanical exercise of applying every update immediately (**National Institute of Standards and Technology, 2022b**). This is particularly important in environments where a poorly tested fix could interrupt critical services. The right question is not only "how severe is the vulnerability?" but also "what is the safest and fastest way to reduce the actual risk for the affected system?"

In many cases, the preferred remediation is a tested software update. However, secure maintenance must also address situations in which an immediate patch is not feasible. Temporary mitigations can include configuration hardening, feature disablement, access-control restrictions, network segmentation, secret rotation, rate limiting, or additional monitoring. These measures should never be treated as invisible substitutes for a real fix. They are time-bounded risk reductions that need explicit ownership, review dates, and documentation of residual exposure. Without those controls, temporary exceptions tend to become permanent weaknesses.

Update assurance is another essential element of secure maintenance. Once a fix has been developed, the organization should verify that the update package is the correct one, that it was tested against relevant regressions, that deployment can be phased or rolled back safely, and that the fix actually reaches the intended environments. NIST highlights the importance of identifying, prioritizing, acquiring, installing, and verifying patches throughout the organization (**2022b**). Verification matters because many maintenance failures do not arise from the absence of a fix, but from inconsistent rollout, incomplete asset coverage, or uncertainty about which versions remain exposed after deployment. A remediation task should not be considered closed until the fix or mitigation is confirmed in the relevant runtime context.

The remediation process also benefits from coordination with vulnerability disclosure channels. CISA's coordinated vulnerability disclosure guidance underscores the importance of clear reporting paths and managed handling of externally reported issues (**Cybersecurity and Infrastructure Security Agency, 2026a**). For software producers, this means maintenance cannot depend only on internal discovery. External researchers, partners, and customers may all be sources of security knowledge, and a mature maintenance program should be able to accept, assess, and respond to that information consistently.

Continuous improvement and learning loops

Secure maintenance becomes strategically valuable when it improves future software, not only current releases. Every significant incident, late-stage vulnerability, emergency patch, or recurring misconfiguration exposes a learning opportunity. If those events are handled only as isolated tickets, the same weakness classes will reappear across future versions. If they are analyzed for root cause and translated into preventive improvements, maintenance becomes a driver of lifecycle maturity.

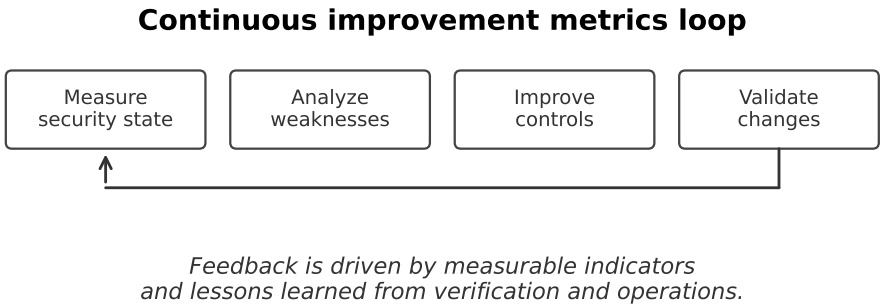

Figure 3.12: Operational evidence translated into analysis, engineering changes, and measurable improvement outcomes.

NIST's incident-response guidance stresses lessons learned and the need to share them as part of continuous cybersecurity improvement (**National Institute of Standards and Technology, 2025**). Lessons learned should not be limited to major breaches. Repeated dependency issues, patch delays, avoidable misconfigurations, weak test coverage for security fixes, and recurring manual exceptions are all indicators that the maintenance process or the upstream engineering process needs refinement. Useful follow-up actions may include updating secure coding standards, tightening deployment policy gates, expanding regression tests, adding exploit-relevant monitoring, or changing supplier requirements for components and services.

Measurement helps make this improvement process credible. Organizations do not need an excessive number of metrics, but they do need a small set that links maintenance effort to risk reduction. Examples include time to remediate

critical findings, age of unresolved exceptions, percentage of supported versions still in operation, coverage of key verification checks during hotfix releases, and rate of repeat defects belonging to the same weakness class. Metrics should support decisions, not create compliance theater. If a measure does not influence prioritization or process design, it is unlikely to improve maintenance in a meaningful way.

OWASP SAMM is helpful here because it frames software assurance as a measurable improvement journey rather than a binary state of compliance (**OWASP Foundation, 2026d**). That perspective is valuable for maintenance programs, which often evolve incrementally. An organization may begin with basic patch handling and later add stronger version governance, more reliable closure verification, more structured incident feedback, and better automation around exceptions and policy enforcement. The goal is not to create a perfect maintenance regime overnight, but to make maintenance more repeatable, visible, and resilient over time.

3.5 Secure Decommissioning and Disposal of Software Components

Secure software lifecycle governance should also define how software components are retired. Risk does not arise only from actively maintained systems, but from half-removed services, abandoned repositories, dormant interfaces, stale service accounts, and unsupported versions that remain reachable long after business ownership has shifted elsewhere. Decommissioning should therefore be treated as a controlled security activity requiring a disposal or transition plan, information preservation decisions, media sanitization, and documented disposition (**Kissel et al., 2008; Ross et al., 2022**).

The core objective of decommissioning is to remove a component without leaving behind hidden attack paths or losing information that must legally, operationally, or contractually be retained. This is especially important when software has reached end of support. CISA's guidance on patches and software up-

dates notes that unsupported or end-of-life software no longer receives normal security updates and therefore creates avoidable exposure if it remains in use (**Cybersecurity and Infrastructure Security Agency, 2023b**). CISA's more recent material on product security bad practices reinforces the same point by treating unsupported products as an avoidable source of risk (**2025**). A mature lifecycle therefore needs a retirement process that is planned, evidenced, and connected to inventory, change records, identity management, archival rules, and environment cleanup.

Planned retirement and controlled service withdrawal

Decommissioning should begin with a formal retirement trigger: end of support, product replacement, consolidation, major architecture change, unacceptable maintenance cost, or a risk decision that continued operation is no longer justified. Once retirement is approved, the first step is not deletion but scoping. Teams need to identify the component versions in use, dependent services, exposed endpoints, linked data stores, scheduled jobs, certificates, update channels, monitoring hooks, and human or machine identities still tied to the retiring element. NIST's SDLC guidance warns that disposed systems or components may remain dormant yet still connected to the infrastructure, creating unnecessary risk if they are overlooked (**Kissel et al., 2008**). In practice, this is one of the main reasons decommissioning fails: organizations remove visible functionality but forget background integrations and residual trust relationships.

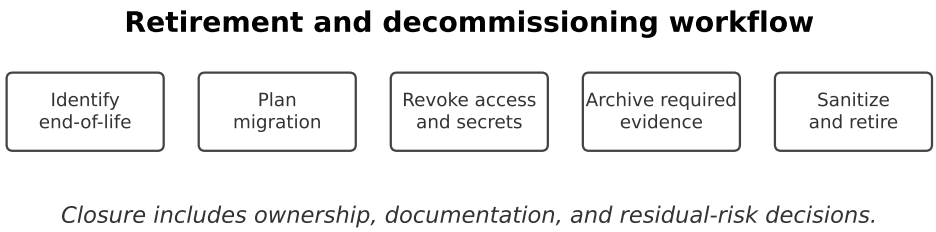

Figure 3.13: Controlled retirement workflow for software components, from decommissioning trigger to closure evidence.

A practical decommissioning plan should therefore define the target state be-

fore anything is disabled. It should state which capabilities are being removed, which data must be migrated or archived, which clients must be redirected, what rollback or contingency path exists if shutdown is not immediately successful, and which artifacts will remain for audit or legal reasons. It should also define a withdrawal sequence so that external exposure is reduced early. For example, public entry points, routing rules, webhooks, API tokens, package publication jobs, and automated service accounts are often better removed before deep infrastructure cleanup begins. OWASP DSOMM describes a defined decommissioning process as a way to ensure that unused applications are removed from the inventory of production components, reducing the chance that obsolete services remain forgotten in the environment (**OWASP Foundation, 2026a**).

Communication is part of this control problem. If consumers, operators, or administrators are not notified of shutdown milestones, they may recreate old access paths or continue using unsupported components through side channels. Decommissioning is therefore not only technical removal but also trust withdrawal: endpoints are retired, identities are revoked, support commitments are closed, and inventories are updated so the organization no longer treats the component as an active security responsibility.

Preservation, sanitization, and identity offboarding

Once the retirement scope is known, the organization must decide what is preserved and what is destroyed. NIST's lifecycle guidance stresses that disposal should preserve vital information, migrate or archive data when required, and sanitize media before reuse or disposal (**Kissel et al., 2008**). The same logic applies to software components. Source code history, release records, audit trails, tickets, configuration baselines, and selected logs may need to be retained for compliance, forensics, contractual support, or future reference. By contrast, cached artifacts, temporary environments, replicated datasets, expired backups, and obsolete deployment packages should not persist indefinitely without an explicit retention basis.

Where data-bearing media or artifacts are involved, sanitization should fol-

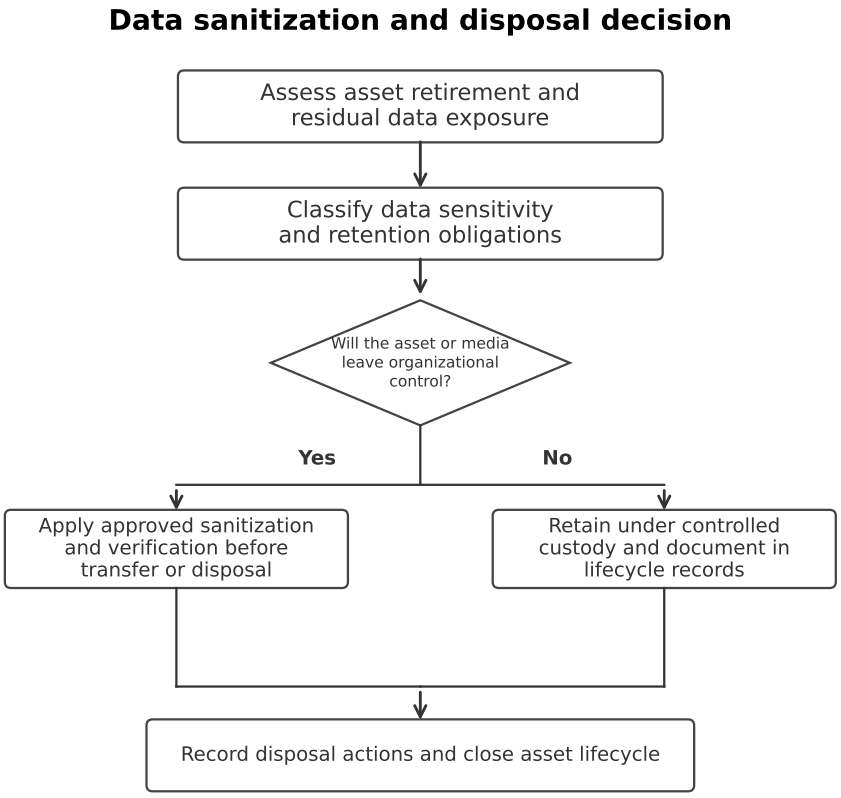

Figure 3.14: Decision flow for preservation, sanitization, credential revocation, and final disposition during software-component retirement.

low an approved method rather than an informal deletion step. NIST SP 800-88 Rev. 2 defines media sanitization as making access to target data infeasible for a given level of effort and ties the choice of technique to information sensitivity, media characteristics, and intended reuse or disposal (**Chandramouli & Hibbard, 2025**). For software retirement, this means that simply deleting a virtual machine, container, repository, or storage volume is not always sufficient if snapshots, persistent disks, exported databases, package caches, or backup copies remain elsewhere. The decommissioning decision must therefore consider the entire artifact chain, not just the primary runtime instance.

Identity cleanup is equally important. If service accounts, API keys, certificates, webhook secrets, signing credentials, or machine-to-machine identities remain active after a component has been retired, they can become a residual attack surface. NIST account-management guidance requires system accounts

to be created, disabled, and removed according to policy, and current OWASP guidance on non-human identities highlights improper offboarding as a recurring risk when deprecated services leave behind dormant access paths (**OWASP Foundation, 2025g; Ross, Pillitteri, et al., 2024**). Accordingly, a complete decommissioning action should include credential revocation, secret rotation where trust boundaries change, and confirmation that automated jobs and integrations can no longer authenticate against retired resources.

Evidence of closure and residual-risk prevention

A component should be considered decommissioned only when closure can be demonstrated. Useful evidence includes updated inventories, removal from routing and discovery systems, revoked certificates and service identities, deletion or archival records for data-bearing assets, termination of CI/CD jobs and update channels, removal from monitoring baselines, and documented exceptions if any retained artifact remains temporarily in place. This type of evidence matters because decommissioning is often judged successful too early, while DNS records, stale images, orphaned backups, or old support interfaces are still present.

For that reason, the final stage should include a short verification period in which teams confirm that no legitimate traffic still depends on the retired component, no new secrets are being issued for it, and no operational runbooks still reference it as an active service. If a replacement system exists, dependency maps, SBOM-related records, and support documentation should be updated so downstream teams are not pointed back toward obsolete versions. Overall, secure decommissioning reduces attack surface by ensuring that retirement is deliberate, traceable, and complete. It closes the lifecycle by combining transition planning, data preservation, sanitization, credential offboarding, and verifiable withdrawal of trust from software components that are no longer meant to operate (**Chandramouli & Hibbard, 2025; Cybersecurity and Infrastructure Security Agency, 2023b; Kissel et al., 2008**).

3.6 Synthesis: Integrating All Lifecycle Controls into a Consistent SSDLC Model

The previous sections have examined security controls as they appear across the lifecycle, from design through retirement. The main lesson is that these activities should not be treated as isolated checkpoints. A coherent SSDLC model requires security objectives to remain traceable from early requirements to decommissioning. Both NIST SSDF and OWASP SAMM support this view by organizing secure development as a lifecycle-wide discipline rather than a collection of phase-specific tasks (**OWASP, 2026; Souppaya et al., 2022**). Lifecycle consistency matters as much as the quality of any single control.

A useful synthesis is therefore to view the SSDLC as a control system with three persistent properties. First, it needs *continuity*: security requirements, architectural decisions, test evidence, release records, operational findings, and retirement actions should remain linked over time instead of being recreated in disconnected documents. Second, it needs *integrity*: artifacts, dependencies, build outputs, deployment states, and operational changes should be protected against unauthorized modification. Third, it needs *feedback*: incidents, vulnerabilities, audit results, and maintenance experience must flow back into design choices, coding standards, test priorities, and procurement decisions. Without these properties, an organization may perform many security tasks while still failing to maintain a secure lifecycle.

A consistent lifecycle model

Figure 3.15 summarizes an integrated SSDLC model. The outer loop represents lifecycle progression, from requirements and architecture to implementation, verification, release, deployment, operations, maintenance, and retirement. The center of the model represents the cross-cutting functions that should remain active throughout: governance, risk management, traceability, automation, and evidence retention.

In practice, this means that each lifecycle phase should both consume and

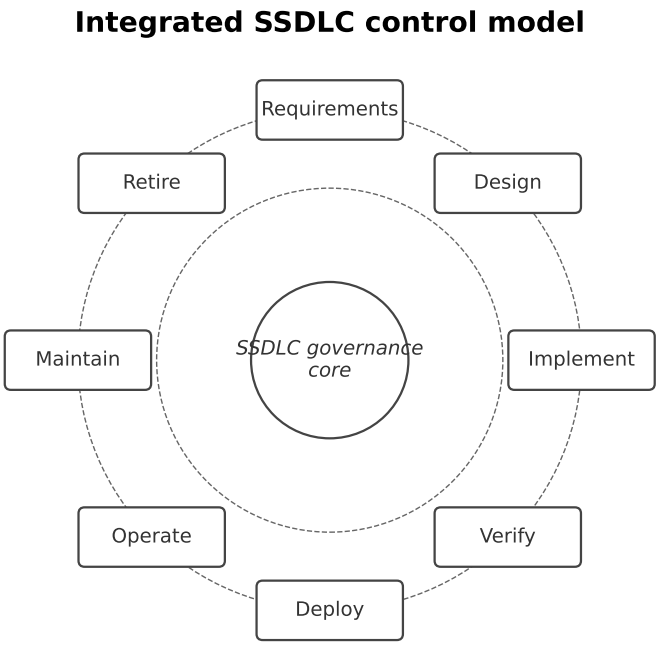

Figure 3.15: Integrated SSDLC model showing lifecycle phases around cross-cutting governance, risk, traceability, automation, and feedback functions.

produce security artifacts. Requirements should express security objectives and acceptance criteria. Architecture should transform those objectives into design constraints, trust boundaries, and risk treatments. Implementation should apply secure coding rules, dependency controls, and code review evidence. Verification should confirm that required controls are present and effective. Release engineering should preserve provenance, integrity, and approval records. Deployment and operations should maintain hardened configurations, observability, and incident readiness. Maintenance should reassess exposure as software, dependencies, and environments evolve. Decommissioning should close the chain by withdrawing trust, sanitizing retained assets when needed, and updating inventories. What makes the model consistent is not that each phase has different controls, but that each phase hands forward structured evidence to the next one.

Cross-cutting control families and feedback loops

An SSDLC also becomes more robust when controls are grouped into a small number of recurring families. In the model proposed here, five families appear across the whole lifecycle: governance and policy, design and implementation assurance, verification and release integrity, operational resilience, and closure and improvement (**Cybersecurity and Infrastructure Security Agency, 2023a; Souppaya et al., 2022**). Lifecycle consistency depends on management decisions as much as on technical controls.

Lifecycle control mapping matrix

Phase	Plan	Build	Verify	Release	Operate	Retire
Requirements	Define	Refine	Trace	Confirm	Review	Close
Identity & access	Scope	Implement	Test	Approve	Monitor	Revoke
Change control	Baseline	Review	Retest	Promote	Record	Archive
Evidence	Criteria	Artifacts	Results	Attest	Audit	Preserve
Supply chain	Select	Track	Scan	Sign	Monitor	Dispose
Response	Prepare	Instrument	Exercise	Escalate	Respond	Learn

Figure 3.16: Illustrative mapping of recurring control families across major SSDLC stages.

The supply-chain perspective is especially important in this synthesis. NIST SP 800-161 treats cybersecurity supply chain risk management as an enterprise-wide activity and integrates it into broader risk management processes rather than limiting it to software acquisition alone (**National Institute of Standards and Technology, 2022a**). Accordingly, third-party risk should not appear only in procurement or SBOM generation. It should influence architecture choices, component selection, build integrity, deployment approval, vulnerability response, and retirement planning. In a mature SSDLC, internal development controls and supply-chain controls are therefore part of the same assurance model.

The same principle applies to learning loops. Operational findings should

not remain inside SOC or operations tooling. Vulnerability trends, misconfiguration patterns, incident root causes, patch latency, and exceptions granted during release or deployment should all feed back into earlier lifecycle decisions. Figure 3.17 illustrates this feedback mechanism. The goal is not only to fix current issues, but to reduce recurrence by changing the conditions that produced them in the first place.

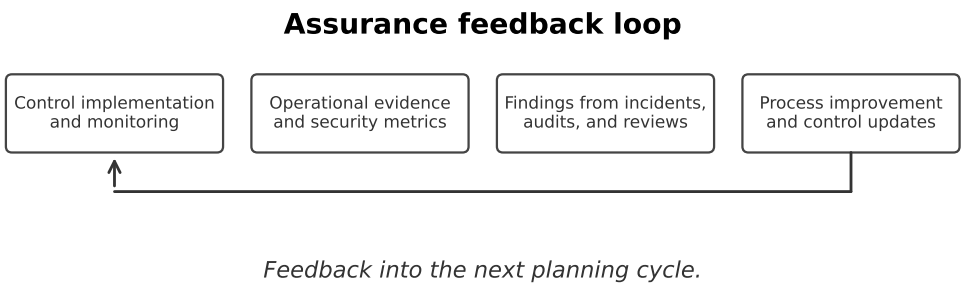

Figure 3.17: Feedback loop linking operational evidence, vulnerability response, design updates, coding guidance, and release controls into continuous lifecycle improvement.

From control accumulation to lifecycle discipline

The most important conclusion of this chapter is that an SSDLC should be judged less by the number of separate security activities it contains than by the consistency with which those activities are connected. Organizations often accumulate tools, scans, checklists, and reviews without creating a common operating model for how evidence is produced, how decisions are recorded, and how lessons learned are propagated forward. A more sustainable approach is to define a limited set of cross-cutting control objectives and then implement them repeatedly, with appropriate depth, in each lifecycle stage.

This view is also compatible with maturity-based improvement. OWASP SAMM is explicitly designed as a measurable improvement model (**OWASP, 2026**), while NIST SSDF is intentionally high-level so that it can be mapped to different development processes and assurance needs (**Souppaya et al., 2022**). The resulting SSDLC model can therefore be described as a risk-informed, evidence-

driven, and continuously improving lifecycle in which all phases are treated as parts of the same security system rather than as separate compliance events.

Conclusion

Summary

This thesis has examined the Secure Software Development Lifecycle as a structured and continuous model for integrating security into the conception, design, implementation, verification, deployment, operation, maintenance, and retirement of software systems. The central argument developed across the preceding chapters is that software security cannot be treated as an isolated technical add-on or as a final compliance checkpoint. Instead, it must be approached as a lifecycle discipline in which security objectives are translated into architectural decisions, development practices, assurance activities, operational controls, and long-term governance mechanisms. In this perspective, the quality of software security depends not only on the correctness of individual controls, but also on the consistency with which those controls are connected across stages.

The first chapter established the conceptual basis for this argument by discussing secure software engineering fundamentals, core security principles, risk-oriented reasoning, security requirements engineering, and the role of regulatory and compliance constraints in shaping early lifecycle decisions. Security was framed not as an absolute state, but as a managed effort to reduce risk to an acceptable level while preserving confidentiality, integrity, availability, accountability, and trustworthiness. This foundation made it possible to analyze security requirements not as static statements, but as drivers of later architectural, implementation, and operational choices.

The second chapter then focused on the design, implementation, verification, and change-control dimensions of the lifecycle. The discussion of architectural

design showed that many relevant security properties become constrained before coding starts. Threat modeling, architectural risk analysis, and the disciplined use of secure patterns were therefore presented as mechanisms for preventing structural weaknesses rather than merely discovering defects after implementation. The section on secure implementation practices emphasized that coding standards, input validation, error handling, data protection, peer review, and automated analysis should be understood as mutually reinforcing activities rather than independent safeguards. Security testing was then positioned as a lifecycle-wide assurance function in which planning, automated testing, selective manual analysis, and traceability to requirements are combined to validate whether intended controls actually operate as expected. Finally, configuration and change management showed that trustworthy software delivery depends on controlled build pipelines, dependency governance, protected repositories, and release processes capable of preserving integrity and accountability from source change to deployable artifact.

The third chapter extended the same logic into deployment and operations. Secure deployment and environment hardening were analyzed as trust-preserving transitions between validated artifacts and production services. Operational controls such as monitoring, logging, auditing, patch management, and incident response were discussed as mechanisms for sustaining security after release rather than as external functions detached from development. The chapter also showed that software supply chain security has become inseparable from modern software assurance because dependencies, package ecosystems, build services, container bases, and update channels all influence the trustworthiness of the final system. Secure maintenance and continuous improvement further demonstrated that software security is not preserved automatically over time; it must be actively maintained through remediation workflows, metrics, feedback loops, and governance. The discussion of decommissioning and retirement completed the lifecycle view by showing that unsupported components, residual identities, retained data, and incomplete disposal can reintroduce risk even when a system is approaching end of life. The final synthesis argued that an effective SSDLC

is best understood as an integrated control model in which governance, design, implementation, verification, deployment, operations, supply chain assurance, maintenance, and retirement are treated as parts of the same security system.

Taken together, the chapters support a clear conclusion: the most mature expression of software security is not the accumulation of tools or isolated best practices, but the creation of a repeatable and evidence-driven lifecycle. This conclusion is consistent with the NIST Secure Software Development Framework, which is explicitly designed to be integrated into different development life cycles, with secure-by-design guidance that places responsibility on software producers to reduce avoidable risk by default, and with OWASP SAMM, which frames software assurance as a measurable and risk-driven improvement process across the full lifecycle. (**Cybersecurity and Infrastructure Security Agency, 2023a; OWASP, 2026; Souppaya et al., 2022**)

Benefits and Limitations of Current SSDLC Practices

One of the main benefits of contemporary SSDLC approaches is that they provide a common structure for aligning technical practices with organizational risk management. Instead of treating security as a collection of exceptional reviews, an SSDLC makes security visible in ordinary engineering activities: requirements analysis, architectural choices, secure coding, testing, release approval, monitoring, maintenance, and retirement. This improves traceability because decisions can be connected to risks, controls, and evidence. It also improves accountability because responsibilities become easier to assign and verify. Frameworks such as NIST SSDF and OWASP SAMM are particularly valuable in this respect because they offer organizations a way to organize secure development activities without prescribing a single methodology or toolchain. (**OWASP, 2026; Souppaya et al., 2022**)

A second benefit is the earlier reduction of avoidable defects. When threat modeling, architectural review, secure design principles, and dependency eval-

uation occur before implementation is complete, many weaknesses can be prevented at lower cost than post-release remediation. Likewise, when repositories, build systems, and release processes are protected as integrity-critical assets, organizations reduce the risk that compromised or unreviewed changes enter production. The practical significance of this benefit is substantial: mature lifecycle controls shift effort from reactive vulnerability handling toward preventive design and controlled delivery. Secure-by-design guidance emphasizes exactly this shift by arguing that security should be built into products as a producer responsibility, rather than delegated to downstream customers after deployment. (**Cybersecurity and Infrastructure Security Agency, 2023a; Souppaya et al., 2022**)

A third benefit lies in resilience and learning. An SSDLC that includes monitoring, patch governance, incident response, and maintenance feedback loops is better positioned to absorb change over time. Vulnerability information, misconfiguration trends, incident findings, and operational exceptions can be used to refine earlier lifecycle stages, thereby reducing recurrence rather than only correcting symptoms. This creates a more sustainable form of assurance because the organization learns from its own evidence. The lifecycle model thus supports not only prevention and detection, but also adaptation. (**OWASP, 2026; Souppaya et al., 2022**)

Despite these benefits, current SSDLC practices also have important limitations. The first is uneven implementation depth. Many organizations adopt the language of secure development without achieving real lifecycle integration. They may perform scans, maintain checklists, or require formal approvals, but still lack clear traceability between risks, requirements, controls, and operational evidence. In such cases, the SSDLC becomes symbolic: activities exist, yet they are weakly connected and provide limited assurance. This gap between formal adoption and effective integration remains one of the most significant practical limitations of current secure development programs. (**OWASP, 2026; Souppaya et al., 2022**)

A second limitation concerns tool dependence. Automation is essential in

modern development environments, especially for code review support, static analysis, dependency monitoring, build integrity, and deployment policy enforcement. However, tools can create an illusion of completeness if their outputs are treated as substitutes for engineering judgment. Architectural weaknesses, contextual authorization failures, insecure trust assumptions, and business-logic flaws often remain difficult to identify through automated scanning alone. As a result, SSDLC maturity still depends on human review, architectural reasoning, and organizational capability, not only on pipeline instrumentation.

A third limitation is the tension between speed, complexity, and control. Modern software ecosystems rely on rapid iteration, cloud-native deployment patterns, large dependency graphs, third-party services, and distributed teams. These features increase development velocity, but they also enlarge the number of identities, interfaces, artifacts, and external trust relationships that must be governed. Supply chain risk management guidance makes clear that these dependencies should be treated as part of enterprise risk rather than as isolated technical details, yet operationalizing this at scale remains difficult for many organizations. (**National Institute of Standards and Technology, 2022a**)

Finally, many SSDLC models remain easier to describe than to measure. It is relatively straightforward to list recommended practices; it is more difficult to demonstrate how well they work in a given environment, how consistently they are applied, and how much risk they actually reduce. For this reason, maturity and effectiveness should not be inferred from the presence of policies alone. They require evidence, metrics, exception handling, and repeated reassessment.

Emerging Trends

Several emerging trends suggest that secure software development will become even more evidence-driven, automated, and ecosystem-aware in the coming years. The first trend is the growing normalization of secure-by-design expectations. Regulatory pressure, public guidance, procurement practices, and customer expectations are increasingly shifting responsibility upstream toward software pro-

ducers. This does not mean that all risk can be eliminated, but it does mean that avoidable insecure defaults, weak initial configurations, and preventable exposure are less acceptable as normal features of software products. The secure-by-design movement reflects this broader shift from reactive patching culture toward producer accountability and safer baseline product behavior. (**Cybersecurity and Infrastructure Security Agency, 2023a**)

A second trend is the consolidation of software supply chain transparency as a standard assurance requirement. SBOM generation, dependency visibility, provenance records, signing, and attestations are becoming more relevant not only for large vendors but also for organizations that consume and integrate software components at scale. The value of this trend is not limited to inventory. Greater transparency supports faster vulnerability response, more reliable impact assessment, and better decision-making concerning component selection, update trust, and release approval. In this sense, software supply chain transparency is evolving from a specialized practice into a broader requirement for lifecycle accountability. (**National Institute of Standards and Technology, 2022a; National Telecommunications and Information Administration, 2021**)

A third trend is the increasing use of automation and policy enforcement across the development and delivery chain. Secure build pipelines, infrastructure-as-code validation, automated testing, repository protection, deployment gating, and machine-readable control evidence all point toward a model in which assurance is generated continuously rather than assembled only at release time. This does not eliminate the need for expertise, but it changes the location of security work by embedding more of it into ordinary engineering workflows. (**Souppaya et al., 2022**)

A fourth trend concerns AI-assisted software development and assurance. The growing use of generative AI for code generation, test generation, documentation support, and security analysis introduces both opportunities and risks. On one side, AI assistance may improve developer productivity and help scale review or testing tasks. On the other, it may also accelerate the production of insecure code, propagate weak patterns, or obscure provenance and accountability if

outputs are accepted uncritically. NIST has already recognized this development by publishing an SSDF profile specific to AI systems and model development, signaling that secure lifecycle principles must adapt to AI-influenced production environments rather than remain limited to traditional software processes. (**Booth et al., 2024; Souppaya et al., 2022**)

Taken together, these trends indicate that future SSDLC practice will likely place increasing emphasis on continuous assurance, verifiable evidence, safer defaults, stronger supply chain guarantees, and the careful governance of AI-assisted engineering activities.

Future Research Directions

Although the SSDLC is now supported by mature frameworks and guidance, several research directions remain open. One important area concerns the measurement of security effectiveness across the lifecycle. Future work should investigate how organizations can define metrics that reflect not only activity completion, but also the quality and impact of lifecycle controls. Examples include the relationship between architectural review quality and downstream defect density, the effect of dependency governance on remediation speed, or the reliability of deployment policy gates in preventing configuration drift.

A second research direction concerns the integration of supply chain assurance into routine engineering decisions. While inventories, SBOMs, provenance models, and signing technologies are advancing, more work is needed to understand how these mechanisms should influence design choices, procurement decisions, release criteria, and maintenance prioritization in practice. In other words, future research should not only refine supply chain artifacts, but also study how they can be operationalized as decision support instruments across the lifecycle.

A third area concerns the human and organizational dimensions of the SSDLC. Secure development is not implemented by frameworks alone. It depends on role clarity, incentives, governance, communication between development and operations teams, and the ability to translate security objectives into normal

engineering behavior. Research on secure development maturity should therefore pay greater attention to socio-technical factors, especially in environments characterized by rapid delivery, outsourcing, platform engineering, and cross-functional product teams.

A fourth research direction involves AI-assisted development. As AI tools become more deeply embedded in coding, testing, and operational decision support, it will become increasingly important to study how these systems affect secure design quality, defect propagation, review reliability, and evidence generation. Future research should explore how human oversight, provenance controls, evaluation methods, and secure development standards can be adapted to maintain trust when some lifecycle activities are partially delegated to AI systems.

In conclusion, the Secure Software Development Lifecycle should not be interpreted as a rigid sequence of isolated security tasks, but as a governance and engineering model for producing software whose security can be reasoned about, verified, operated, improved, and eventually retired in a controlled manner. The most important lesson emerging from this thesis is that lifecycle security is achieved when security requirements, design decisions, implementation discipline, verification evidence, operational controls, supply chain assurance, maintenance practices, and retirement procedures are connected into a coherent system. Where these elements remain fragmented, security remains reactive and incomplete. Where they are integrated, software assurance becomes more robust, more transparent, and more sustainable over time.

Bibliography

Black, P. E., Badger, L., Guttman, B., Fong, E., Souza, J., Chang, R., & Skok, W. (2021). *Guidelines on minimum standards for developer verification of software* (NIST Interagency Report No. 8397). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8397

Booth, H., et al. (2024). *Secure software development practices for generative ai and dual-use foundation models: An ssdf community profile* (tech. rep. No. NIST SP 800-218A). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-218A

Chandramouli, R. (2022, March). *Implementation of devsecops for a microservices-based application with service mesh* (tech. rep. No. NIST SP 800-204C). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/204/c/final

Chandramouli, R., & Hibbard, E. A. (2025). *Guidelines for media sanitization* (SP 800-88 Rev. 2). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://doi.org/10.6028/NIST.SP.800-88r2

Chandramouli, R., Kautz, F., & Torres-Arias, S. (2024a). *Strategies for the integration of software supply chain security in devsecops ci/cd pipelines* (tech. rep. No. NIST SP 800-204D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-204D

Chandramouli, R., Kautz, F., & Torres-Arias, S. (2024b). *Software supply chain security in devsecops ci/cd pipelines* (SP 800-204D). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/204/d/final

Cybersecurity and Infrastructure Security Agency. (2023a). *Secure by design*. CISA. Retrieved February 28, 2026, from https://www.cisa.gov/sites/default/files/2023-10/SecureByDesign_1025_508c.pdf

Cybersecurity and Infrastructure Security Agency. (2023b). *Understanding patches and software updates*. Retrieved February 28, 2026, from https://www.cisa.gov/news-events/news/understanding-patches-and-software-updates

Cybersecurity and Infrastructure Security Agency. (2024a). *Federal government cybersecurity incident and vulnerability response playbooks*. CISA. Retrieved February 28, 2026, from https://www.cisa.gov/resources-tools/resources/federal-government-cybersecurity-incident-and-vulnerability-response-playbooks

Cybersecurity and Infrastructure Security Agency. (2024b). *Secure by demand guide: How software customers can drive a secure technology ecosystem*. CISA. Retrieved February 28, 2026, from https://www.cisa.gov/sites/default/files/2024-08/SecureByDemandGuide_080624_508c.pdf

Cybersecurity and Infrastructure Security Agency. (2024c). *Software acquisition guide for government enterprise consumers: Software assurance in the cyber-supply chain risk management lifecycle*. CISA. Retrieved February 28, 2026, from https://www.cisa.gov/sites/default/files/2024-07/PDM24050%20Software%20Acquisition%20Guide%20for%20Government%20Enterprise%20ConsumersV2_508c.pdf

Cybersecurity and Infrastructure Security Agency. (2025). *Product security bad practices*. Retrieved February 28, 2026, from https://www.cisa.gov/resources-tools/resources/product-security-bad-practices

Cybersecurity and Infrastructure Security Agency. (2026a). *Coordinated vulnerability disclosure program*. Retrieved February 28, 2026, from https://www.cisa.gov/resources-tools/programs/coordinated-vulnerability-disclosure-program

Cybersecurity and Infrastructure Security Agency. (2026b). *Known exploited vulnerabilities catalog*. Retrieved February 28, 2026, from https://www.cisa.gov/known-exploited-vulnerabilities-catalog

Google Open Source. (2026). *Oss-fuzz documentation*. Retrieved February 28, 2026, from https://google.github.io/oss-fuzz/

in-toto Authors. (2025). *In-toto*. Retrieved February 28, 2026, from https://in-toto.io/

Johnson, A., Dempsey, K., Ross, R., Gupta, S., & Bailey, D. (2019). *Guide for security-focused configuration management of information systems* (tech. rep. No. NIST SP 800-128). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-128.pdf

Kazman, R., Klein, M., & Clements, P. (2000). *Atam: Method for architecture evaluation* (tech. rep.). Software Engineering Institute, Carnegie Mellon University. Retrieved February 28, 2026, from https://www.sei.cmu.edu/documents/629/2000_005_001_13706.pdf

Kissel, R., Stine, K., Scholl, M., Rossman, H., Fahlsing, J., & Gulick, J. (2008). *Security considerations in the system development life cycle* (SP 800-64 Rev. 2). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://doi.org/10.6028/NIST.SP.800-64r2

Kubernetes Authors. (2024). *Pod security admission*. Retrieved February 28, 2026, from https://kubernetes.io/docs/concepts/security/pod-security-admission/

Kubernetes Authors. (2025). *Security checklist*. Retrieved February 28, 2026, from https://kubernetes.io/docs/concepts/security/security-checklist/

Microsoft. (2022). *Microsoft threat modeling tool overview*. Retrieved February 28, 2026, from https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool

Microsoft. (2025a). *Security development and operations overview*. Microsoft Learn. Retrieved February 28, 2026, from https://learn.microsoft.com/

en - us / compliance / assurance / assurance - security - development - and - operation

Microsoft. (2025b). *Security development lifecycle (sdl) practices*. Microsoft Security Engineering. Retrieved February 28, 2026, from https://www.microsoft.com/en-us/securityengineering/sdl/practices

MITRE. (2025a). *Cwe-1008: Architectural concepts*. Retrieved February 28, 2026, from https://cwe.mitre.org/data/definitions/1008.html

MITRE. (2025b). *Cwe-701: Weaknesses introduced during design*. Retrieved February 28, 2026, from https://cwe.mitre.org/data/definitions/701.html

National Institute of Standards and Technology. (2006). *Sp 800-92: Guide to computer security log management*. NIST. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/92/final

National Institute of Standards and Technology. (2011). *Sp 800-137: Information security continuous monitoring (iscm) for federal information systems and organizations*. NIST. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/137/final

National Institute of Standards and Technology. (2016). *Guide to data-centric system threat modeling* (tech. rep.). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/154/ipd

National Institute of Standards and Technology. (2021). *Source code security analyzers*. NIST. Retrieved February 28, 2026, from https://www.nist.gov/itl/ssd/software-quality-group/source-code-security-analyzers

National Institute of Standards and Technology. (2022a). *Cybersecurity supply chain risk management practices for systems and organizations* (SP 800-161 Rev. 1). NIST. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/161/r1/final

National Institute of Standards and Technology. (2022b). *Sp 800-40 rev. 4: Guide to enterprise patch management planning*. NIST. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/40/r4/final

National Institute of Standards and Technology. (2025). *Sp 800-61 rev. 3: Incident response recommendations and considerations for cybersecurity risk management*. NIST. Retrieved February 28, 2026, from https://csrc.nist.gov/pubs/sp/800/61/r3/final

National Security Agency & Cybersecurity and Infrastructure Security Agency. (2022, August). *Kubernetes hardening guidance* (tech. rep.). National Security Agency, Cybersecurity, and Infrastructure Security Agency. Retrieved February 28, 2026, from https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF

National Telecommunications and Information Administration. (2021). *The minimum elements for a software bill of materials (sbom)*. U.S. Department of Commerce. Retrieved February 28, 2026, from https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom

Okun, V., & Fong, E. (2015). Fuzz testing for software assurance. *Crosstalk*. Retrieved February 28, 2026, from https://www.nist.gov/publications/fuzz-testing-software-assurance

Open Source Security Foundation. (2025). *Openssf scorecard*. Retrieved February 28, 2026, from https://scorecard.dev/

OWASP. (2026). *Owasp samm model overview* [Last access: 28/02/2026]. https://owaspsamm.org/model/

OWASP Foundation. (n.d.-a). *Ci/cd security cheat sheet*. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). *Vulnerable dependency management cheat sheet*. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Vulnerable_Dependency_Management_Cheat_Sheet.html

OWASP Foundation. (2020). *Owasp vulnerability management guide*. Retrieved February 28, 2026, from https://owasp.org/www-project-vulnerability-management-guide/OWASP-Vuln-Mgm-Guide-Jun01-2020.pdf

OWASP Foundation. (2025a). *A06:2021 – vulnerable and outdated components*. Retrieved February 28, 2026, from https://owasp.org/Top10/2021/A06_2021-Vulnerable_and_Outdated_Components/

OWASP Foundation. (2025b). *A06:2025 insecure design*. Retrieved February 28, 2026, from https://owasp.org/Top10/2025/A06_2025-Insecure_Design/

OWASP Foundation. (2025c). *Cryptographic storage cheat sheet*. OWASP Cheat Sheet Series. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

OWASP Foundation. (2025d). *Error handling cheat sheet*. OWASP Cheat Sheet Series. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

OWASP Foundation. (2025e). *Input validation cheat sheet*. OWASP Cheat Sheet Series. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

OWASP Foundation. (2025f). *Key management cheat sheet*. OWASP Cheat Sheet Series. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Key_Management_Cheat_Sheet.html

OWASP Foundation. (2025g). *Nhi1:2025 improper offboarding*. Retrieved February 28, 2026, from https://owasp.org/www-project-non-human-identities-top-10/2025/1-improper-offboarding/

OWASP Foundation. (2025h). *Owasp application security verification standard 5.0*. Retrieved February 28, 2026, from https://owasp.org/www-project-application-security-verification-standard/

OWASP Foundation. (2025i). *Owasp dependency-check*. Retrieved February 28, 2026, from https://owasp.org/www-project-dependency-check/

OWASP Foundation. (2025j). *Owasp secure by design framework*. Retrieved February 28, 2026, from https://owasp.org/www-project-secure-by-design-framework/

OWASP Foundation. (2025k). *Owasp software component verification standard*. Retrieved February 28, 2026, from https://owasp.org/www-project-software-component-verification-standard/

OWASP Foundation. (2025l). *Password storage cheat sheet*. OWASP Cheat Sheet Series. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (2025m). *Secure code review cheat sheet*. OWASP Cheat Sheet Series. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Secure_Code_Review_Cheat_Sheet.html

OWASP Foundation. (2025n). *Threat modeling cheat sheet*. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

OWASP Foundation. (2025o). *Web security testing guide*. Retrieved February 28, 2026, from https://owasp.org/www-project-web-security-testing-guide/

OWASP Foundation. (2026a). *Defined decommissioning process*. Retrieved February 28, 2026, from https://dsomm.owasp.org/activity-description?activityName=Defined+decommissioning+process&dimension=Build+and+Deployment&level=2&subDimension=Deployment&uuid=da4ff665-dcb9-4e93-9d20-48cdedc50fc2

OWASP Foundation. (2026b). *Docker security cheat sheet*. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

OWASP Foundation. (2026c). *Logging cheat sheet*. OWASP. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

OWASP Foundation. (2026d). *Owasp software assurance maturity model*. Retrieved February 28, 2026, from https://owaspsamm.org/model/

OWASP Foundation. (2026e). *Secrets management cheat sheet*. Retrieved February 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Ross, R., Pillitteri, V., et al. (2024). *Protecting controlled unclassified information in nonfederal systems and organizations* (SP 800-171 Rev. 3). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://doi.org/10.6028/NIST.SP.800-171r3

Ross, R., Winstead, M., & McEvilley, M. (2022). *Engineering trustworthy secure systems* (SP 800-160 Vol. 1 Rev. 1). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://doi.org/10.6028/NIST.SP.800-160v1r1

Scarfone, K., Jansen, W., & Tracy, M. (2008, July). *Guide to general server security* (tech. rep. No. NIST SP 800-123). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-123.pdf

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). *Technical guide to information security testing and assessment* (tech. rep. No. NIST SP 800-115). National Institute of Standards and Technology. Retrieved February 28, 2026, from https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-115.pdf

Sigstore Project. (2025). *Sigstore overview*. Retrieved February 28, 2026, from https://docs.sigstore.dev/about/overview/

SLSA. (2025). *Supply-chain levels for software artifacts*. Retrieved February 28, 2026, from https://slsa.dev/

Software Engineering Institute. (2020). *Sei cert coding standards wiki*. Carnegie Mellon University. Retrieved February 28, 2026, from https://www.sei.cmu.edu/library/sei-cert-coding-standards-wiki/

Souppaya, M., Morello, J., & Scarfone, K. (2017, September). *Application container security guide* (tech. rep. No. NIST SP 800-190). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-190

Souppaya, M., Scarfone, K., & Dodson, D. (2022, February). *Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (tech. rep. No. NIST SP 800-218). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-218

TUF Authors. (2026). *The update framework*. Retrieved February 28, 2026, from https://theupdateframework.io/